Debian 12 alatt Apache2 webszerver telepítése és üzemeltetése (6×45 perc) – PHP-FPM és MariaDB integrációval

2. foglalkozás (45 perc): **Apache2 webszerver telepítése Debian 12-n, alapkonfiguráció, tűzfal és naplózás**

Oktatási cél és előfeltételek

- **Cél:** A diákok telepítsék és indítsák el az Apache2 webszervert a Debian 12 rendszeren. Megismerik az Apache alapbeállításait, a webszerver könyvtárstruktúráját és a kiszolgálás alapjait. Beállítják a tűzfalat a webszerver portjának engedélyezésére. Megtanulják a naplófájlok helyét és azok olvasását az Apache hibakereséséhez.
- **Előfeltételek:** Debian 12 szerver a korábbi foglalkozás szerint telepítve, hálózatról elérhető IP-vel. Alapszintű parancssori ismeretek (apt csomagkezelő, szolgáltatások indítása). Ajánlott: internetkapcsolat a csomagok letöltéséhez vagy előzetesen letöltött csomagok/tárhely elérhetősége LAN-on.

Tanári magyarázat és demonstráció

Apache webszerver áttekintése: Az Apache HTTP Server a világ egyik legelterjedtebb webszervere, moduláris felépítésű, nyílt forráskódú szoftver[11]. Debian 12 csomagtárolóiban a legfrissebb stabil Apache 2.4 verzió érhető el. Az Apache a LAMP környezet "A" betűje (Linux + Apache + MySQL/MariaDB + PHP).

Apache telepítése Debian alatt: Debianban az Apache telepítése egyszerű az APT csomagkezelővel. Rendszergazdaként futtassuk:

```
sudo apt update          # csomaglista frissítése (ha még nem történt meg)
sudo apt install apache2
```

Ez felteszi az apache2 csomagot és függőségeit. A telepítés után az Apache szolgáltatás automatikusan elindul és bekapcsolódik a rendszerindításkor (systemd service)[12]. Ellenőrizzük a státuszt:

```
sudo systemctl status apache2
```

A kimenetnek aktív (running) állapotot kell mutatnia. Továbbá, Debian alatt az Apache alapértelmezés szerint a 80-as HTTP porton figyel (és a 443-as HTTPS porton, ha bekapcsoljuk SSL-t).

Az Apache alapértelmezett weboldal tesztelése: A Debian Apache telepítése létrehoz egy alapértelmezett weboldalt. Kérjük le ezt a böngészőben: nyissuk meg a host gépen a VM IP címét (pl. **http://192.168.1.100/**). Ha mindent jól csináltunk, megjelenik az Apache „It works!" vagy Debian test oldal (HTML tartalom a /var/www/html/index.html-ből). Így meggyőződhetünk arról, hogy a szerver fut és kiszolgálja a kéréseket.

Tűzfalbeállítások (UFW) – portok megnyitása: Ha a Debian szerveren tűzfal fut (alapesetben Debian 12-n nincs aktív iptables szabály vagy UFW, hacsak nem telepítettük), érdemes bevezetni a tűzfal fogalmát. Telepítsük és konfiguráljuk az UFW-t (Uncomplicated Firewall) a könnyű kezelés érdekében:

```
sudo apt install ufw
sudo ufw allow 'Apache'
sudo ufw enable
sudo ufw status
```

Az `'Apache'` profil az UFW-ben előre definiált szabály, ami a 80/tcp (HTTP) portot engedélyezi (és általában a 443/tcp-t is)[13]. A `status` parancs mutatja, hogy pl. **Apache ALLOW** szerepel a szabályok között. (Ha nem lenne profil, `sudo ufw allow 80/tcp` is megteszi ugyanazt.) Fontos kiemelni, hogy a tűzfal a bejövő forgalmat szabályozza – LAN környezetben is legyenek tisztában a diákok azzal, hogy a webszerver portját engedélyezni kell, különben nem érhető el a szolgáltatás.

Apache konfigurációs fájlok és könyvtárak: Debian alatt az Apache konfiguráció **modulárisan** van felépítve, az összes config az `/etc/apache2` könyvtárban található[14]. Néhány fontosabb elem:

- - **`apache2.conf`:** fő konfigurációs fájl. Ebben globális beállítások vannak, és betölt más konfigurációkat is (`Include` utasításokkal)[15][16].
- - **`ports.conf`:** a hallgató portok definíciója (pl. `Listen 80`).
- - **`sites-available/`** és **`sites-enabled/`:** Virtuális host (webhely) konfigurációk helye. A *sites-available* könyvtárban lévő minden `.conf` fájl egy webhely konfigurációját tartalmazza, de csak az *sites-enabled* alá bekapcsolt (symlinkelt) konfigurációk aktívak. Az aktiválást segítő parancs az **a2ensite**, a letiltást az **a2dissite** végzi. Alapértelmezésben van egy `000-default.conf` nevű site, amely a port 80-ra a /var/www/html könyvtárat szolgálja ki.
- - **`mods-available/`** és **`mods-enabled/`:** Apache modulok be- és kikapcsolására szolgáló helyek. A mod-available könyvtárban találhatók a modul leíró fájljai (`*.load` és `*.conf`), az enabled könyvtárban pedig szimbolikus linkek jelzik az

engedélyezett modulokat. Az **a2enmod** parancs hoz létre linket (engedélyez modult), az **a2dismod** pedig eltávolítja azt.

- - **`conf-available/`** és **`conf-enabled/`:** Egyéb általános konfigurációs fragmentumok (nem vhost-specifikus). Itt pl. az `serve-cgi-bin.conf` vagy a PHP-FPM integrációs confja található. Ezeket az **a2enconf/a2disconf** kapcsolja.

Mutassuk meg a diákoknak a `tree /etc/apache2 -L 1` paranccsal a struktúrát, vagy egyszerűen magyarázzuk táblán. Hangsúlyozzuk: Debian csomagoknál a moduláris megközelítés van érvényben – nem egy hatalmas httpd.conf fájlt szerkesztünk mindent egyben, hanem külön fájlokban dolgozunk, és a segédparancsokkal aktiváljuk azokat. Ez áttekinthetőbb és verziókezelhetőbb.

Alapvető direktívák és beállítások: Nézzünk bele a `/etc/apache2/sites-available/000-default.conf` fájlba (pl. `sudo nano`-val vagy `cat`-tel). Itt látunk egy `<VirtualHost *:80>` blokkot, benne:

- `DocumentRoot /var/www/html` – az alapértelmezett webkönyvtár.

- `ServerAdmin webmaster@localhost` – admin email (hibaoldalakon jelenhet meg).

- `ErrorLog ${APACHE_LOG_DIR}/error.log` és `CustomLog ${APACHE_LOG_DIR}/access.log combined` – a naplófájlok helye és formátuma. (Az APACHE_LOG_DIR a `/var/log/apache2` alapértelmezés szerint.)

- `<Directory /var/www/html>` blokk: fontos beállításokat tartalmaz a könyvtárra:

  - `Options Indexes FollowSymLinks` – engedélyezi a könyvtárindex megjelenítést (ha nincs indexfájl) és a symlinkek követését.

  - `AllowOverride None` – tiltja a .htaccess fájlokban levő utasítások érvényesülését alapértelmezetten (később ezt WordPress-nél állítjuk majd `All`-ra, hogy a .htaccess működjön).

  - `Require all granted` – bárki hozzáférhet (engedélyezve a tartalom megtekintése).

Magyarázzuk el, hogy a `DocumentRoot` utáni könyvtárban található fájlokat szolgálja ki az Apache. Az alapértelmezett index.html ezért jelenik meg. Ha azt a fájlt módosítják vagy törlik, az kihat a megjelenésre.

Apache szolgáltatás kezelése: Systemd-n fut, így:

- `sudo systemctl restart apache2` – Apache újraindítása (pl. config módosítás után).

- `sudo systemctl enable apache2` – biztosítja az automatikus indulást (ez már megtörtént telepítéskor).

- `sudo systemctl stop apache2` – leállítás. - `sudo apachectl configtest` – konfigurációs szintaxis ellenőrzése. Ezt mindig érdemes futtatni mielőtt újraindítjuk a szervert egy módosítás után, hogy lássuk van-e elírás.

Naplófájlok megtekintése: Az Apache két fő naplót vezet:

- **Hozzáférési napló (access log):** `/var/log/apache2/access.log` – minden HTTP kérés egy sora bekerül ide (kliens IP, időpont, HTTP metódus, URL, státuszkód stb.). A formátuma Debianban a "combined" (kombinált) formátum, ami a szabványos mezőkön felül a referert és user agentet is tartalmazza.

- **Hibanapló (error log):** `/var/log/apache2/error.log` – ide kerül minden hibaüzenet, CGI/PHP hiba, vagy akár induláskor figyelmeztetések.

Mutassuk meg, hogyan lehet a `tail -f` parancs segítségével folyamatosan figyelni a logokat:

```
sudo tail -f /var/log/apache2/access.log
```

Nyissák meg a böngészőt és frissítsék az oldalakat – látni fogják a megjelenő sorokat (IP, GET kérés, HTTP 200 kód stb.)[25][26]. Az error.log-ba nézzünk bele pl. `tail /var/log/apache2/error.log`. (Jelenleg valószínűleg üres vagy csak server indítási infókat tartalmaz.) Hangsúlyozzuk, hogy hibakeresésnél ez a két fájl lesz a legjobb barátjuk – pl. egy 403 Forbidden vagy 500 Internal Server Error okát az error.log-ból lehet megtudni.

Diákfeladatok lépésről lépésre

1. **Apache telepítése:** A diákok a saját Debian VM-jükön telepítsék az Apache2 csomagot. Használják a `sudo apt install apache2` parancsot. Figyeljék meg a konzolon a kimenetet (szolgáltatás indítása).
2. **Szolgáltatás ellenőrzése:** Futassák a `systemctl status apache2` parancsot. Keressék meg benne a „active (running)" részt. Kérdés: melyik felhasználó alatt fut az Apache? (A status kimenetben vagy a process listában látható, hogy **www-data** userként indul – ezt magyarázzuk, hogy biztonsági okból van külön webszerver felhasználó.)
3. **Böngészős teszt:** A host Windows 10 gépen nyissanak meg egy böngészőt, és írják be a Debian VM IP-címét (amit az előző óra végén statikusan beállítottak). A böngészőben megjelenő alapértelmezett oldalról készítsenek képernyőképet vagy írják le mit látnak („It works!" üzenet, Debian logó stb.).
4. **HTML módosítás:** A diákok lépjenek be a Debianban a `/var/www/html` könyvtárba: `cd /var/www/html`. Listázzák ki a tartalmát (`ls -l`). Itt látniuk kell az `index.html` fájlt. Másolják át biztonság kedvéért:

```
sudo cp index.html index.html.bak
sudo nano index.html
```

Szerkesszék a HTML fájlt úgy, hogy az „It works” helyett saját nevüket vagy valami egyedi szöveget írnak a `<h1>` tag-ek közé. Mentsék a fájlt, majd frissítsék a böngészőt. **Megjelent az új tartalom?** Ezzel tapasztalják, hogy az Apache a fájlrendszerből szolgálja ki a tartalmat.

5. **UFW tűzfal használata:** Ha az UFW még nincs telepítve, telepítsék: `sudo apt install ufw`. Engedélyezzék a webforgalmat: `sudo ufw allow 'Apache'` (vagy `Apache Full`, ami a 80 és 443 portot is nyitja)[27]. Kapcsolják be a tűzfalat: `sudo ufw enable`. Ellenőrizzék a státuszt. Jegyezzék meg, mi történik, ha a tűzfalban letiltanák a 80-as portot (ilyenkor a weboldal nem lenne elérhető).
6. **Logfájlok megfigyelése:** Indítsák el két terminálablakban:
7. Az egyikben: `sudo tail -f /var/log/apache2/access.log`
8. A másikban: `sudo tail -f /var/log/apache2/error.log`
   Majd a host gépen frissítsék többször az oldalakat, próbáljanak nem létező URL-t is (pl. http://IP/xyz). Figyeljék meg, milyen bejegyzések keletkeznek az access.log-ban (GET kérések, HTTP kódok). A nem létező URL próbánál nézzék meg, hogy a hozzáférési naplóban 404-es kód jelenik meg, az error.log-ban pedig bejegyzés utal a hiányzó fájlra (file not found). Ez segít megérteni a naplók szerepét.

Tipikus hibák és megoldásaik

- **Csomagtelepítési hibák:** Ha a telepítésnél hálózati gond miatt sikertelen a letöltés, győződjünk meg, hogy van internetelérés. Ha proxy kell, azt apt-ben be kell állítani. LAN környezetben a tanár előkészíthet helyi csomagtükröt. Ha apt cache van, használjuk azt.
- **Apache nem érhető el a böngészőből:** Ennek több oka lehet:
- A VM nem jó hálózatot használ (pl. Default Switch, ami nem engedi a külső hozzáférést). Ezt orvosoljuk external switch használatával[6].
- A Windows host saját tűzfala blokkolja a forgalmat. (Ha a Windows gépen külön tűzfal szabály kell, azt is állítsuk – pl. engedélyezzünk bejövő kapcsolatot a VM IP címére a 80-as porton. Ez belső hálón általában nem gond, de domain/network profile függő.)
- Nincs elindítva az Apache vagy összeomlott. Ezt a diák ellenőrizheti a status paranccsal. Ha nem fut, `sudo systemctl start apache2`.
- **„Forbidden” hiba az oldal helyett:** Ha a diák módosította a `/var/www/html` jogosultságait vagy tartalmát rosszul (pl. törölte az index.html-et és nincs index file, de a szerver nem listázhat, mert DirectoryIndex nem talál semmit és Indexes tiltva van), 403 Forbidden jelenhet meg. Nézzük meg az error.log-ot, ami valószínűleg permission denied-ot jelez. Megoldás: állítsuk vissza a jogosultságokat (fájlok www-data tulajdon, 644, könyvtár 755). Alapértelmezésben a `/var/www` és alatta lévő fájlok tulajdonosa **www-data**. Ha a szerkesztés root-ként történt, megeshet, hogy átírta a fájl tulajdonosát root-ra. Javítás: `sudo chown -R www-data:www-data /var/www/html`.

- **Test oldal cache miatt nem frissül:** Ritka, de ha a diák nem látja a módosított tartalmat, lehet böngésző cache. Nyomjanak Ctrl+F5-öt (teljes frissítés). Ez nem Apache hiba, de érdemes megjegyezni.

Összefoglalás és kérdések

Összegzés: A második foglalkozáson a tanulók üzembe helyezték az Apache webszervert. Megtanulták, hogyan telepíthető egy szolgáltatás Linuxon apt segítségével, hogyan futtatják és állítják meg a systemd szolgáltatásokat. Megértették a webszerver alapértelmezett működését és azt, hogy a webszerver a fájlrendszer adott könyvtárából szolgálja ki a kéréseket (DocumentRoot). Gyakorolták a tűzfalbeállításokat UFW-vel, és belekóstoltak a logelemzésbe – látták, hogy a naplók segítségével nyomon követhető minden kérés és hiba.

Kérdések: - Melyik parancsot használjuk egy szolgáltatás (pl. apache2) újraindítására Linuxon?
- Hol található az Apache alapértelmezett webszerver gyökérkönyvtára Debian alatt, és mely fájl szolgálja ki a főoldalt?
- Mi a jelentősége az Apache **sites-enabled** és **sites-available** könyvtárainak? Hogyan lehet egy konfigurációt engedélyezni?
- Milyen adatokat tárol az access.log és az error.log? Hol találhatóak ezek?
- Hogyan engedélyeznél a tűzfalon egy új hálózati portot UFW használatával? Mondj példát.

---

3. foglalkozás (45 perc): **Apache konfiguráció mélyítése – fontos direktívák, DocumentRoot módosítása**

Oktatási cél és előfeltételek

- **Cél:** A résztvevők megértsék az Apache konfigurációs állományainak testreszabását. Elsajátítják, hogyan lehet a webhely gyökérkönyvtárát átállítani egy másik helyre, és megtanulják az ehhez kapcsolódó engedélyezési és modul-beállítási teendőket. Ismerjék meg az Apache fontosabb konfigurációs direktíváit (pl. ServerName, DirectoryIndex, AllowOverride, Options stb.), és értsék azok hatását.
- **Előfeltételek:** Az Apache már fut a Debian rendszeren (előző óra). Alapvető szövegszerkesztési ismeret terminálban (nano). Minimális Linux jogosultsági ismeretek (fájlok tulajdonosa, chmod, chown). Ajánlott: a diák felhasználója sudo-jogos, vagy root felhasználó elérhető.

Tanári magyarázat és demonstráció

Apache fontosabb direktívái és funkcióik:

- **ServerName:** A virtuális host azonosítására szolgál – megadja azt a domain nevet, amelyhez a konfiguráció tartozik. Ha nincs beállítva, induláskor figyelmeztetés keletkezik

(a 000-default.conf esetén tipikus a „Could not reliably determine the server's fully qualified domain name” üzenet). Most még lokális IP alapján érik el a szervert, de ha később hostneveket használunk (pl. saját domain vagy hosts fájlos név), ezt beállítjuk.

- **DirectoryIndex:** Az Apache mely fájlokat keresse automatikusan, ha egy könyvtár URL-jét kérik le. Alapértelmezés: `index.html` (Debianon a `/etc/apache2/mods-enabled/dir.conf` tartalmazza, tipikusan `DirectoryIndex index.html index.cgi index.pl index.php index.xhtml index.htm`). Ezt azért fontos tudni, mert ha statikus oldalt átnevezünk mondjuk home.html-re, akkor 404-et kapunk index helyett, vagy directory listing jelenhetne meg, ha engedélyezve volna.

- **Options:** Már láttuk, például *Indexes* (könyvtárlista engedélyezése) vagy *FollowSymLinks* (szimbolikus link követése). Van még pl. **ExecCGI** (CGI futtatás engedélyezése), **Includes** (Server Side Includes engedélyezése), stb. Jellemzően egy `<Directory>` blokkban állítjuk be adott könyvtárra. - **AllowOverride:** Meghatározza, hogy a .htaccess fájlok milyen direktívákat írhatnak felül. `None` esetén semmit; `All` esetén bármit, vagy finomhangolható pl. `AuthConfig`, `FileInfo` stb. modulok szerint. Ezt a WordPress-nél majd *All*-ra kell állítanunk, hogy a WP által generált .htaccess-ben levő URL átírás szabályok működjenek.

- **Require [all granted|denied]:** Hozzáférés szabályozás. A `Require all granted` jelenti, hogy bárki hozzáférhet (nem IP-re korlátozunk). Lehetne pl. `Require ip 192.168.1.0/24` a csak belső hálóra korlátozáshoz, vagy autentikációs modulokkal `Require valid-user` stb.

- **LogLevel:** Ezzel állíthatnánk az Apache naplózási szintjét (debug, info, warn, error…). Alapértelmezésben `warn`. Ha problémát keresünk, felemelhetjük debug-ra, de ekkor nagyon sok log keletkezik.

- **ServerTokens és ServerSignature:** Biztonsági okokból szokták változtatni. A ServerTokens Minimal/Prod beállítással lehet csökkenteni, hogy az Apache ne árulja el a verzióját a HTTP headerben. A ServerSignature Off letiltja, hogy a generált hibaoldalak alján megjelenjen az Apache verzió infó.

Ezen direktívák kapcsán mutassunk gyakorlati példát: nyissuk meg az `/etc/apache2/apache2.conf`-ot. Itt a fájl végén látható, hogy **IncludeOptional sites-enabled/*.conf**, stb., tehát ebből is látszik a modularitás[16]. Keressük meg a `<Directory /var/www/>` részt. Debian alapbeállításban így néz ki:

```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Magyarázzuk, hogy ez a /var/www gyökerére vonatkozik rekurzívan, kivéve ahol máshogy felülbírálják (pl. a sites-available confban lehet specifikusabb /var/www/html). Fontos, hogy ha a DocumentRoot-ot áthelyezzük máshova, erről a Directory beállításról gondoskodni kell.

DocumentRoot átállítása – miért és hogyan: Tegyük fel, hogy nem a `/var/www/html`-t szeretnénk használni webszerver könyvtárnak. Lehet oka:

- Szeretnénk a felhasználók **home könyvtárában** tartani a webfájlokat (pl. /home/student/www).

- Külön meghajtóra tennénk a web tartalmat (pl. /srv/www vagy /var/www/mysite). - Biztonsági megfontolásból chroot-szerű környezetet alakítunk ki.

Most egy egyszerű eset: a DocumentRoot-ot átállítjuk egy egyedi könyvtárra. Demonstráljuk mondjuk azt, hogy a web root a jövőben a **/home/student/weboldal** könyvtár lesz. Lépések:

1. **Új könyvtár létrehozása:**

```
mkdir /home/student/weboldal
```

Tegyük bele gyorsan egy index file-t:

```
echo "<h1>Saját weboldal</h1>" > /home/student/weboldal/index.html
```

Jelenleg ez a könyvtár a `student` user tulajdona (mivel ő hozta létre). Ahhoz, hogy az Apache ki tudja szolgálni, vagy átállítjuk a jogosultságokat, vagy engedélyezzük a userdir modult (következő óra témája), de itt inkább úgy vesszük, hogy a webszervert futtató user (www-data) fogja olvasni ezt.

2. **Apache konfiguráció módosítása:** Másolhatjuk a default vhost configot egy újhoz, de egyszerűség kedvéért módosítjuk a meglévőt.
Nyissuk meg: `sudo nano /etc/apache2/sites-available/000-default.conf`
Keressük meg a `DocumentRoot /var/www/html` sort, és írjuk át:

```
DocumentRoot /home/student/weboldal
```

Emellett érdemes a fájl elején egy `ServerName sajatweb.local` (vagy hasonló) sort is megadni, hogy ne legyen figyelmeztetés. (Ez egy tetszőleges név, később hosts fájlban ráirányíthatjuk.)
Fontos: a fájl végén vagy máshol lehet még `<Directory>` direktíva a régi útra. Ebben a default conf-ban nincs külön, de az apache2.conf-ban van egy általános /var/www. Ezért inkább itt, a vhost confban adjunk hozzá egy megfelelő Directory bejegyzést az új útra:

```
<Directory /home/student/weboldal>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Ez megengedi, hogy az új gyökérkönyvtár tartalmát elérjék. Mentés után lépjünk ki.

3. **Jogosultságok beállítása:** Ahogy említettük, a `weboldal` könyvtár és benne az index.html most student tulajdon. Az Apache user (www-data) nem biztos, hogy hozzáfér. Ilyenkor két lehetőség van: vagy a `www-data` usert betesszük a student csoportjába és megfelelő group permeket adunk, **vagy** egyszerűbb: átadjuk a fájlokat a www-data felhasználónak tulajdonba.
Oktatási környezetben egyszerűbb:

```
sudo chown -R www-data:www-data /home/student/weboldal
sudo chmod -R 755 /home/student/weboldal
```

Ezzel minden fájlt könyvtárat olvashatóvá teszünk a webszerver számára[28]. (Magyarázzuk: 755 = tulajdonos írhat, mindenki olvashat/végezhet.) Továbbá nagyon fontos, hogy a /home/student könyvtár is beléphető legyen a webszerver által. A Linux nem engedi a `/home/student/weboldal` olvasását, ha a /home/student mappa x jogosultsága nincs meg másoknak. Adjunk legalább keresési engedélyt:

```
chmod 711 /home/student
```

Így a `student` home könyvtárba listázni nem tud a webszerver, de be tud lépni a benne levő `weboldal` könyvtárba[29].

4. **Konfiguráció teszt és Apache újraindítás:**

```
sudo apachectl configtest
```

Ha `Syntax OK`, indítható újra:

```
sudo systemctl restart apache2
```

Figyeljünk, van-e hibaüzenet a restartnál vagy az error.log-ban.

5. **Tesztelés böngészőben:** Nyissuk meg újra a szerver IP-címét a böngészőben. Mit látunk? Az új „Saját weboldal” feliratot. Tehát sikerült áthelyezni a DocumentRoot-ot. Ha nem jelenne meg, nézzük meg az Apache error.log-ot.

Lehetséges hibák:

- „Permission denied” – akkor a jogosultságot nem állítottuk be jól (pl. /home/student maradt 700).

- 500 Internal Error – akkor valószínű config szintaxis hiba, de a configtest ezt kiszűrte volna.

- 404 Not Found – lehet elírtuk a DocumentRoot path-t vagy rossz helyre tettük az index.html-t.

Egyedi webhely konfiguráció külön fájlban (opcionális bemutatás): Megemlíthetjük, hogy általában nem a 000-defaultot módosítjuk, hanem létrehozunk új site configot. Például készíthettünk volna egy `sajatweb.conf`-ot a sites-available-ben ezzel a tartalommal:

```
<VirtualHost *:80>
    ServerName sajatweb.local
    DocumentRoot /home/student/weboldal

    <Directory /home/student/weboldal>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/sajatweb_error.log
    CustomLog ${APACHE_LOG_DIR}/sajatweb_access.log combined
</VirtualHost>
```

A külön error/access log beállítás csak esztétika (külön fájlba írja a domainhez). Ezután `sudo a2ensite sajatweb.conf` és `sudo a2dissite 000-default.conf` parancsokkal lehetne ezt aktiválni és a defaultot kikapcsolni[30][31]. Végül restart. Ezt haladóbb körben csinálják meg, a lényeg most a DocumentRoot váltás megértése volt. (Említsük: a Debian default 000-default.conf direkt nem ad meg ServerName-et, hogy bármilyen hostname esetén ez szolgáljon. Ha sajátot veszünk fel, érdemes a defaultot kilőni, különben host név hiányában is mehetne rá forgalom.)

Diákfeladatok lépésről lépésre

1. **Új webkönyvtár létrehozása:** Minden diák találjon ki egy könyvtárnevet a saját home-jában, pl. `weboldal_{sajatnev}`. Hozza létre:

   ```
   mkdir ~/weboldal_demo
   echo "Hello, demo weboldal!" > ~/weboldal_demo/index.html
   ```

   Majd listázza: `ls -l ~/weboldal_demo`. (Ellenőrizzék, hogy van index.html és tartalmaz valami szöveget.)
2. **Apache beállítása az új DocumentRoot-ra:** Válasszuk azt a módszert, hogy a default site-ot módosítjuk (mert egyszerűbb a diákoknak). Nyissák meg sudo-val a `/etc/apache2/sites-available/000-default.conf` fájlt. Keressék meg a `DocumentRoot` sort és cseréljék ki a saját home-os könyvtáruk elérési útjára, pl. `/home/student/weboldal_demo`.

3. Adjanak hozzá (vagy szerkesszék, ha van) egy Directory blokkot, ami az új útra vonatkozik, `Require all granted` és hasonló opciókkal (a tanár segít, kivetítheti mit kell beírni, vagy a segédanyagban ott a példa).
4. Mentsék el a módosításokat.
5. **Jogosultságok beállítása:** A diákok adják ki a megfelelő chown és chmod parancsokat a saját könyvtárukra. Pl.:

```
sudo chown -R www-data:www-data /home/student/weboldal_demo
sudo chmod -R 755 /home/student/weboldal_demo
sudo chmod 711 /home/student
```

   (Figyelem: ha valaki root-ként hozta létre a könyvtárat az előző lépésben sudo-val, akkor különösen fontos a chown, mert akkor root a tulaj.)
6. **Konfiguráció tesztelése és újraindítás:** Futtassák: `sudo apachectl configtest`. Ha hibát jelez, javítsák a fájlt (valószínűleg elütés vagy rossz szögletes zárójel). Ha OK, akkor `sudo systemctl restart apache2`.
7. **Tesztelés:** Nyissák meg böngészőben újra a szerver IP-t. Elvárt eredmény: az általuk létrehozott "Hello, demo weboldal!" vagy egyedi szöveg jelenjen meg, azaz az Apache már az új helyről szolgál.
8. Ha működik, gratuláljunk nekik – saját könyvtárukból megy a webszerver.
9. Ha nem, nézzék meg az error.log-ot: `tail /var/log/apache2/error.log`. Gyakori hiba, ha kimaradt a /home mappa jogosultság (Permission denied a logban). Ezt orvosolják a `chmod 711 /home/<username>` paranccsal, majd újraindítással.
10. Másik tipikus hiba: config fájlban elgépelt útvonal, pl. `/home/studen/weboldal_demo` (hiányzó t betű). Az error.log-ban ilyenkor access log-ban 404-et lát, error.log-ban nem találja az index fájlt az útvonalon. Javítsák a configban és reload.
11. **(Haladóbb) Hostnév beállítás hosts fájlban:** Ha szeretnék, a diákok a saját gépük hosts fájljába felvehetnek egy bejegyzést, hogy a webszerverüket ne IP-vel kelljen elérni. Például: nyissák meg jegyzettömbbel rendszergazdaként a `C:\Windows\System32\drivers\etc\hosts` fájlt a Windows 10-en, és írják bele:

```
192.168.1.100   sajatweb.local
```

   (A megfelelő IP-vel és kitalált névvel.) Mentsék, majd a böngészőben az **http://sajatweb.local/** címet megnyitva is ugyanazt kell lássák (feltéve, hogy az Apache configban a ServerName vagy a default catch-all kiszolgálja). Ez a lépés opcionális, de segít a későbbi virtualhost témánál.

Tipikus hibák és megoldásaik

- **Elírás a konfigurációban:** Ha a diák configtest futtatása nélkül indítja újra az Apache-ot, és kimaradt egy `</Directory>` vagy hasonló, akkor az Apache nem indul el. Ilyenkor a `systemctl status apache2` pirossal jelez hibát. Meg kell nézni a

hibanaplót, amely megmondja, melyik sorban van szintaxis hiba. Szerencsére a configtest az ilyeneket jelzi (pl. "AH00526: Syntax error..."). Rögzüljön bennük: mindig futtassanak configtest-et nagyobb változtatás után![32].

- **Jogosultsági problémák:** Tipikus, hogy kihagyják a home könyvtár +x engedélyét másoknak. Ekkor 403 Forbidden lesz a böngészőben. Az error.log világosan kiírja: „Permission denied: access to /~user/ denied" vagy a konkrét path. A megoldás a chmod 711 a home könyvtárra[33] (vagy alternatív, hogy az Apache user felvétel a home könyvtár csoportjába és setfacl... de ez bonyolultabb).
- **Elfelejtett Directory bejegyzés:** Ha átállítják a DocumentRoot-ot, de nem adnak hozzá megfelelő `<Directory>` engedélyezést, akkor szintén 403 Forbidden vagy 500 lehet. Az Apache 2.4-ben a default policy a `<Directory /> Require all denied`. Így ha az új út nincs kifejezetten engedélyezve, tiltott lesz. Ezt megelőztük azzal, hogy betettük a configba a Directory blokkot. Ha kimaradt, gyorsan pótolják és reload[34][35].
- **Nem megfelelő modulok:** Ebben a gyakorlatban nem nagyon érinti őket, de pl. ha CGI-s tartalmat akarnának futtatni, kellene az `ExecCGI` opció vagy a mod_cgid modult betölteni. Vagy ha .htaccess-zel akarnak URL-t írni, kell a mod_rewrite. Ezek hiányában nem működnének dolgok.
- A modulok engedélyezésére példa: mod_rewrite bekapcsolása: `sudo a2enmod rewrite` majd restart. Ezt a következő óra (WP telepítés) előtt amúgy is meg kell tennünk, ott visszatérünk rá.

Összefoglalás és kérdések

Összegzés: A harmadik órán a tanulók elmélyítették Apache konfigurációs ismereteiket. Képesek már alap virtuális host konfigurációt értelmezni és módosítani. Áthelyeztük a webszerver gyökérkönyvtárát egy tetszőleges helyre, ami rávilágított a jogosultságok és a konfiguráció összefüggéseire. Szó esett több fontos Apache beállításról és a .htaccess szerepéről (AllowOverride), mely a következő órákban a PHP és WordPress kapcsán előtérbe kerül.

Kérdések: - Milyen lépéseket kell megtenni ahhoz, hogy az Apache webszerver egy másik könyvtárból szolgálja ki a weboldalt?
- Mi a `AllowOverride` direktíva szerepe, és mi történik, ha None helyett All-ra állítjuk?
- Hogyan lehet engedélyezni, hogy az Apache listázza egy könyvtár tartalmát, ha hiányzik az indexfájl? Melyik direktíva szabályozza ezt?
- Miért kellett módosítani a home könyvtár jogosultságait, amikor a DocumentRoot-ot a felhasználó home-jába tettük?
- Sorolj fel legalább három Apache modult vagy funkciót, amit külön engedélyezni vagy konfigurálni lehet (pl. mod_rewrite, mod_userdir, CGI, SSL stb.), és írd le röviden, mire valók!

---

4. foglalkozás (45 perc): **PHP-FPM integráció az Apache2-vel, PHP beállítások és modulok kezelése**

Oktatási cél és előfeltételek

- **Cél:** A hallgatók telepítsék a PHP-t és ismerjék meg a PHP kétféle Apache integrációs módját (mod_php vs. PHP-FPM). Beállítják az Apache-ot úgy, hogy a PHP-FPM szolgáltatáson keresztül dolgozza fel a PHP fájlokat. Megtanulják a PHP konfigurációs fájljának (php.ini) néhány fontosabb beállítását (pl. error display fejlesztői környezetben, upload limit, timezone). Gyakorolják a PHP modulok telepítését (pl. MySQL driver).
- **Előfeltételek:** Telepített Apache webszerver Debian 12-n. A diák tudja szerkeszteni az Apache configot. Alapszintű PHP ismeret (szintaxis, mire való a phpinfo). Minimális elmélet: mi az a webes backend script.

Tanári magyarázat és demonstráció

PHP futtatása webszerveren – mod_php vs. PHP-FPM: A PHP egy szerveroldali szkriptnyelv, gyakori webszerver kiegészítő. Apache-pal két módon integrálható: - **mod_php:** egy Apache modul (libapache2-mod-php), ami a httpd folyamaton belül futtatja a PHP-t. Ehhez az Apache-nak a régi **prefork MPM**-et kell használnia. Egyszerű, de nagy terhelésnél lassúbb és több memóriát fogyaszt. - **PHP-FPM (FastCGI Process Manager):** egy különálló démonként futtatja a PHP értelmezőt (pool of worker processes), és az Apache a **mod_proxy_fcgi** modulon keresztül továbbítja a PHP kéréseket neki. Ez modernebb megközelítés, jobban skálázódik (az Apache használhatja az event MPM-et), és jellemzően gyorsabb is (kb. 3× teljesítménynövekedést is jelenthet magas terhelésnél a mod_php-hez képest)[36]. Manapság ez az ajánlott.

Mi a tanfolyam során a PHP-FPM megoldást valósítjuk meg. Debian 12-ben a PHP 8.2 érhető el stabilként, ennek FPM módját fogjuk használni.

PHP és PHP-FPM telepítése:

```
sudo apt install php php-fpm
```

Ez telepíti a PHP értelmezőt parancssori használattal (php-cli), a PHP-FPM szolgáltatást és néhány alapmodult. (Ellenőrizhetjük a verziót: `php -v`.) A telepítés után a **php8.2-fpm** systemd szolgáltatás automatikusan elindul. Ellenőrizzük:

```
sudo systemctl status php8.2-fpm
```

legyen „active (running)”.

Apache beállítása a PHP-FPM használatára: Alapból, ha telepítettük a `libapache2-mod-php` csomagot, akkor a mod_php modul betöltődne és rögtön működne a PHP (prefork MPM-mel). Mi azonban modul nélkül, FPM-el akarjuk.

Ehhez:

- Győződjünk meg, hogy a mod_php **nincs** engedélyezve. (Ha valaki véletlen felrakta a php-mysql csomaggal együtt a mod-php-t is, tiltsuk le: `sudo a2dismod php8.2`. Ezt a configtest se jelezné hibának, de feleslegesen bent lehet.)

- Engedélyezni kell az Apache-nál a FastCGI proxy modult és kapcsolódó komponenst:

```
sudo a2enmod proxy_fcgi setenvif
```

(A proxy_fcgi modul biztosítja, hogy az Apache továbbítja a kérést a háttér FastCGI-nak. A setenvif modult a PHP config fragment használja.) Debianban gyakran a telepítő csomag magától engedélyezte ezt – ellenőrizzük az `/etc/apache2/mods-enabled` alatt van-e `proxy_fcgi.load` link. Ha apt-gettel telepítettük a php-fpm csomagot, valószínű automatikusan behúzta.)

- Továbbá Debianban van egy kész konfigurációs fájl a PHP-FPM integrációhoz: **`/etc/apache2/conf-available/php8.2-fpm.conf`**. Ezt engedélyezzük:

```
sudo a2enconf php8.2-fpm
```

Ez a config beállítja, hogy a `.php` kiterjesztésű fájlokra az Apache ne önmaga próbálja meg kiszolgálni, hanem továbbítsa a PHP-FPM socketnek[37][38]. (Konkrétan tartalmaz egy `<FilesMatch \.php$>` blokkot, benne a `SetHandler "proxy:unix:/run/php/php8.2-fpm.sock|fcgi://localhost/"` utasítással[38][39]. Ez azt mondja: minden .php fájl kérést irányítson a php8.2-fpm.sock Unix domain socketre.)

Ezen a ponton emlékeztethetjük őket: a Linux socket olyan, mint egy „fájl”, amin keresztül folyik a kommunikáció a két folyamat között. A socket elérési útja `/run/php/php8.2-fpm.sock`, a PHP-FPM ezt hozta létre.

- Miután bekapcsoltuk a conf-ot, újra kell indítani az Apache-ot: `sudo systemctl reload apache2` (vagy restart).

Teszteljük a PHP működését: Hozzunk létre egy PHP oldalt a DocumentRoot-ban:

```
sudo nano /home/student/weboldal_demo/info.php
```

Írjuk bele:

```
<?php
phpinfo();
?>
```

Mentsük. (Ha nem a home-os utat használjátok, akkor /var/www/html-ben is lehet, attól függ hol tartotok, a lényeget magyarázzuk.) Most a böngészőben nyissuk meg: **http://<szerver-ip>/info.php**. Ennek hatására az Apache a PHP-FPM-nek adja a kérést, az pedig legenerál egy PHP Info oldalt, amit visszakapunk. Ezen az oldalon rengeteg információ látható a PHP beállításairól és moduljairól[40][41]. (Mutassuk rá: a "Loaded

Configuration File" mező mutatja, melyik php.ini-t használja, pl. `/etc/php/8.2/fpm/php.ini`. Továbbá a „Server API” sorban „FPM/FastCGI” szerepel, jelezve, hogy nem mod_php, hanem FPM. Ezt hasonlítsuk össze, ha valaki esetleg futtatná CLI-ben a php -i kimenetet, ott „CLI” lenne.)

PHP-FPM pool beállítások (érdekesség): A PHP-FPM process manager több folyamatot futtat a kérések kiszolgálására. A konfigurációját a `/etc/php/8.2/fpm/pool.d/www.conf` fájl tartalmazza. Megnyithatjuk és megmutathatjuk, hogy:

- `listen = /run/php/php8.2-fpm.sock` – tehát itt definiálták a socketet, amin figyel[42]. Lehetne 127.0.0.1:9000 is, de Debian a socketet használja.

- `pm = dynamic` – a process manager módja (dinamikusan indítja a worker folyamatokat)[43]. - `pm.max_children = …` – max. hány PHP worker mehet (hard limit). - `pm.start_servers`, `pm.min_spare_servers`, `pm.max_spare_servers` – induló, minimális, maximális idle worker szám[43]. Ezeket lehet finomhangolni erőforráshoz igazítva. Most egy fejlesztői gépen nem nyúlunk hozzá, de tudjanak róla. Ha sok diák terheli a webszervert egyszerre (teszem fel egy közös nagy szerveren), akkor itt lehet skálázni (pl. több gyerek process, ha erős a gép). Minden változtatás után: `sudo systemctl restart php8.2-fpm` kell.

PHP konfiguráció – php.ini lényeges opciók: A PHP fő konfigurációs fájlja (FPM esetén) a `/etc/php/8.2/fpm/php.ini`. (Ha mod_php lenne, akkor az apache2/php.ini lenne a relevant, de moduláris csomagnál külön ini van CLI-hez, FPM-hez stb.) Néhány fontos beállítás, amit érdemes kiemelni:

- **display_errors:** Alapértelmezésben Off a produkciós konfigurációban (főleg FPM). Fejlesztői környezetben érdemes On-ra tenni, hogy a hibák a böngészőben látszódjanak. Ezt tegyük meg: keressük meg a php.ini-ben (`sudo nano /etc/php/8.2/fpm/php.ini`, `/display_errors` keresés). Állítsuk `display_errors = On`. (Production environmentben maradjon Off, hiszen biztonsági kockázat a hibaüzenetek kiloggolása user felé.)

- **memory_limit:** Maximális memória egy PHP process számára. Alapértelmezés pl. 128M vagy 512M. Ha nagy file-okkal dolgoznak, emelni lehet, de figyelni kell a szerver memóriájára. Most nem módosítjuk.

- **upload_max_filesize és post_max_size:** File feltöltési méretkorlát. Alapértelmezés általában 2M upload_max és 8M post_max. Ha a diákok majd WordPress médiát töltenek fel és nagyobb, találkozni fognak ezzel a korláttal. Mutassuk meg, hol van (keressük `upload_max_filesize`), és lehet egy gyakorlati feladat lesz: növeljék 16M-re mindkettőt. (Ne felejtsék el, a post_max_size-nak >= upload_max-nak kell lennie.)[44].

- **date.timezone:** Nagyon fontos a PHP figyelmeztetések elkerülése végett – ha nincs beállítva, a PHP figyelmeztet, hogy nem tudja a zónaidőt. Kérjük beállítani pl. `date.timezone = "Europe/Budapest"`. (Keressünk rá a php.ini-ben, vegyük ki a pontosvesszőt, állítsuk be.)

A php.ini módosítások után újra kell indítani a PHP-FPM-et, hogy érvényre jussanak:

```
sudo systemctl restart php8.2-fpm
```

(Ezt mindenképp tegyük meg, különben a következő lépésnél nem látszik a változás.)

PHP modulok (kiterjesztések) telepítése: A PHP funkciói modulárisak. Alapból jó pár modul települt (PHP Info oldalon látszik a listájuk). De pl. a MariaDB-hez való kapcsolódáshoz kell a **mysqli** vagy **PDO MySQL** modul. Ezt Debianban külön csomag biztosítja: **php8.2-mysql**. Telepítsük:

```
sudo apt install php8.2-mysql
```

Ez felrakja a MySQL/MariaDB driver modulokat. Mivel FPM-ről van szó, a modul telepítésekor keletkezik egy `/etc/php/8.2/mods-available/pdo_mysql.ini` és `mysqli.ini`, és ezekhez symlink is jön automatikusan a fpm/conf.d-ben. Tehát valószínűleg az apt postinst már aktiválta, de ha nem, akkor a `phpenmod mysqli` parancs segítene. (A phpenmod/phpdismod parancsok modulok gyors bekapcsolására valók CLI-n).
Ellenőrizzük: futtassuk újra a `phpinfo()` oldalt (frissítsük). Most már a *mysqli* szekció is benne lesz, jelzi hogy loaded. Ugyanígy lehet telepíteni egyéb modulokat is. Népszerű modulok: `php8.2-gd` (képmanipuláció), `php8.2-curl`, `php8.2-intl`, `php8.2-xml`, `php8.2-zip`, stb. A WordPress-hez tipikusan kell majd a PHP XML, JSON, ctype, imagine modul – szerencsére ezek nagy része alapból bent van vagy a meta csomaggal jött.

Megjegyzés: Az alábbi paranccsal egyszerre több gyakori modult is feltehettünk volna:

```
sudo apt install php8.2-{mysql,curl,gd,intl,xml,zip}
```

Ez rövidítve telepít egy csomó hasznos modul csomagot[45]. (A kód {} kiterjesztés egy bash trükk, nem kell részletezni, de bemutathatjuk.)

Visszatérés az Apache-hoz – modulok engedélyezése: Mielőtt rátérnénk a WordPress telepítésre (következő óra), fontos, hogy az Apache-nál **engedélyezzük a `rewrite` modult**, mert a WP szép permalinkeket .htaccess-szel csinál majd. Ezt most tegyük meg:

```
sudo a2enmod rewrite
sudo systemctl reload apache2
```

Így a mod_rewrite betöltődik. Még nem használjuk, de később észrevétlenül szükséges lesz. (Ha valaki elfelejti, majd a WP admin figyelmeztet, de megelőzzük.)

Diákfeladatok lépésről lépésre

1. **PHP telepítése:** Adjanak ki: `sudo apt install php php-fpm`. Kövessék a terminál kimenetét. Majd nézzék meg a PHP verziót: `php -v`. Kiírja pl. PHP 8.2.x és a CLI módról infót. Ez jelzi, hogy a parancssori PHP működik.
2. **Apache beállítása PHP-FPM-re:**

3. Engedélyezzék a modulokat: `sudo a2enmod proxy_fcgi setenvif` (ha ezt írja, hogy már engedélyezve van, az nem baj).
4. Engedélyezzék a PHP configot: `sudo a2enconf php8.2-fpm`.
5. Indítsák újra az Apache-ot: `sudo systemctl restart apache2`. (Vagy reload, de biztos, ami biztos restart, hogy modul betöltődés frissüljön.)
6. Ellenőrizzék az Apache státuszát, nincs-e hiba. (Nincs, de jó gyakorlat.)
7. **PHP működésének tesztje:** Hozzanak létre egy `info.php`-t a web rootban (akár a korábbi saját könyvtárukban):

```
echo "<?php phpinfo(); ?>" | sudo tee
/home/student/weboldal_demo/info.php
```

   Majd a host gépen böngésszék meg: http://<ip-cím>/info.php. Lássák a PHP Infó oldalt. Keressenek rajta pl. „Loaded Configuration File" sort és írják ki, hol van a php.ini.
8. **PHP ini szerkesztése:** Nyissák meg a FPM php.ini-t: `sudo nano /etc/php/8.2/fpm/php.ini`. Kérjük meg őket, hogy keressék meg és módosítsák az alábbiakat:
9. `display_errors = On` (fejlesztéshez kapcsolják be a hibák kijelzését)
10. `date.timezone = "Europe/Budapest"` (komment eltávolítása és beállítás)
11. `upload_max_filesize = 16M` és `post_max_size = 16M` (növeljük a feltöltési limitet 16 MB-ra)
    Mentsék a fájlt. (Ezekkel a módosításokkal egyelőre nincs látványos hatás, de a WP-nél hasznos lesz. A display_errors-t tesztelni lehet: írjanak egy szintaktikailag hibás PHP-t és nézzék meg hogy kiírja-e a hibát. Ez opcionális.)
12. **PHP-FPM újraindítása:** A php.ini változtatások életbe léptetéséhez futtassák: `sudo systemctl restart php8.2-fpm`. (Ezt sokan elfelejtik, hangsúlyozzuk.)
13. **PHP Info ellenőrzése:** Frissítsék a böngészőben a phpinfo oldalát. Kereséssel nézzék meg:
14. „display_errors" értékét (Local Value már On kell legyen).
15. „upload_max_filesize" új értékét (16M).
16. „date.timezone" beállítást (Europe/Budapest).
    Ezzel igazolják, hogy sikerült a php.ini módosítása.
17. **PHP modul telepítése:** Telepítsék a MySQL/MariaDB modult: `sudo apt install php8.2-mysql`. (Ha szeretnének, telepíthetnek többet is egy sorban, pl. gd, zip, de a lényeg legalább a mysql menjen.)
    A telepítés után (apt automatikusan újraindíthatja a FPM-et, de nem baj, ha nem, mi megtehetjük) nézzék meg újra a phpinfo-t a „mysqli" vagy „mysqlnd" szóra keresve. Látszódnia kell a modulnak (egy külön részben, vagy az összefoglaló elején a „Configure Command" résznél a sok `--with-mysqli` opció). **Kérdés**: mi történne, ha nem telepítenénk ezt a modult, és a PHP-ben megpróbálnánk kapcsolódni

MariaDB-hez? (Válasz: Fatal error, undefined function mysqli_connect – mert hiányzik a modul.)

18. **Apache mod_rewrite engedélyezése:** Mivel a következő órán szükség lesz rá, futtassák: `sudo a2enmod rewrite` és utána `sudo systemctl restart apache2`. (Ezt esetleg a tanár is elintézheti központilag, de jó gyakorlás nekik. Ha valaki megkérdezi miért kell: a WordPress majd .htaccess-t használ URL átíráshoz, amihez a mod_rewrite modul kell az Apache-ban.)

Tipikus hibák és megoldásaik

- **PHP fájl letöltődik a böngészőben, nem fut le:** Ez azt jelenti, hogy az Apache nem küldi a PHP-FPM-nek a kérést, hanem simán kiszolgálja a .php fájlt statikusként. Ennek oka tipikusan az, hogy kimaradt a PHP-FPM conf engedélyezése, vagy nem indítottuk újra rendesen. Megoldás: `a2enconf php8.2-fpm` és restart apache[46][38]. Ha ez megtörtént, de még mindig nem megy, nézzük a `/etc/apache2/conf-enabled/php*` fájlt, benne a FilesMatch szabályt. (Ügyeljünk, hogy a .php fájl eléréséhez legyen engedélyezve. Ha valaki .phps kiterjesztést használ, azt nem viszi át – ez nem gond.)
- **500 Internal Server Error PHP futtatáskor:** Ilyenkor az Apache error.log ad infót. Lehet például, hogy a PHP-FPM nem fut. Nézzük meg: `systemctl status php8.2-fpm`. Ha leállt, indítsuk el. Vagy config hiba miatt nem indul – pl. ha valaki elrontotta a www.conf vagy php.ini-t. Az error.log-ban a FastCGI kommunikációs hiba (`proxy:unix:/run/php...` nem érhető el) utal erre. Megoldás: config visszaállítás, service restart.
- **PHP-FPM socket jogosultság hiba:** Haladóbb, de megemlíthető: a /run/php/php8.2-fpm.sock socket fájlt a www-data user hozza létre, és az Apache is www-data-ként olvassa, így rendben. Ha azonban valaki root-ként próbálta meghívni vagy megváltoztatta a pool usert, lehet mismatch. Ezt a pool configban lehetne állítani (listen.owner/group). Normál Debian esetben nem kell hozzányúlni, de ha mégis permission denied lenne, ellenőrizni kell, hogy az Apache és FPM ugyanazzal a userrel kommunikál.
- **PHP modul nem töltődik be:** Ha apt-tal telepítettük, általában automatikusan betöltődik (phpenmod). Ha mégsem, használjuk a `sudo phpenmod modulnev` parancsot, majd restart FPM. (Ezt akkor kellhet, ha egy modult kézzel raktunk be vagy ki akarunk kapcsolni valamit, pl. xdebug diákoknál nem kell, phpdismod xdebug.)
- **PHP-fájlból nem látszanak a hibák:** Ha valaki hibás kódot ír és csak egy üres fehér oldal jelenik meg, az azért van, mert a hibakijelzés lehet Off. Mi bekapcsoltuk display_errors-t. Ha mégsem jelenne meg, ellenőrizzük, hogy a script elején nincs-e `error_reporting(0)` vagy valami, illetve hogy a változtatást követően újraindították-e a FPM-et. Fejlesztéskor a hibaüzenet láthatósága segít. Productionben majd ki kell kapcsolni (de azt majd éles környezetben).
- **Böngésző cache zavar:** Ha például az info.php frissítése után is a régi értéket látják, lehet, hogy a böngésző nem frissít. Ctrl+F5.

Összefoglalás és kérdések

Összegzés: A negyedik órán integráltuk a PHP-t a webszerverrel, méghozzá a korszerű PHP-FPM módszerrel. A diákok megértették, hogy a PHP most külön folyamatként fut, és az Apache ehhez továbbítja a kéréseket (FastCGI). Beállítottuk a fejlesztéshez hasznos PHP opciókat és modulokat telepítettünk, különös tekintettel a MariaDB kapcsolat moduljára, mely a közelgő adatbázis használathoz kell. Szó esett a PHP konfigurációs fájlról és arról, hogyan kezelhetik a hibajelzéseket. **Ezzel a LAMP környezet majdnem teljes:** Linux + Apache + MariaDB + PHP mind telepítve és konfigurálva van. A következő lépés a gyakorlati alkalmazás, egy WordPress weboldal telepítése, ami mindezeket használja.

Kérdések: - Mi a különbség a mod_php és a PHP-FPM között architekturálisan, és miért preferáljuk az utóbbit?
- Milyen Apache modult kellett engedélyeznünk ahhoz, hogy az Apache a PHP-FPM-hez továbbítsa a PHP kéréseket?
- Hol található a PHP konfigurációs fájlja a rendszereden, és hogyan lép érvénybe egy ottani módosítás?
- Hogyan tudsz telepíteni egy új PHP modult (pl. hogy a PHP tudjon képeket kezelni GD könyvtárral)? Mutasd be a lépéseket apt-tal.
- Mi történik, ha egy PHP szkriptben szintaktikai hiba van? Hogyan jelenik ez meg a böngészőben fejlesztői vs. éles beállítás esetén?
- Az Apache és PHP összekapcsolásánál mi célt szolgál a `.sock` fájl, és milyen jogosultságokkal bír?

---

5. foglalkozás (45 perc): **Virtuális hosztok és több webhely kiszolgálása Apache alatt**

Oktatási cél és előfeltételek

- **Cél:** A diákok megtanulják, mik azok a virtuális hostok (VirtualHosts) az Apache-ban, és hogyan konfigurálhatnak több webhelyet ugyanazon a szerveren. Gyakorolják egy egyedi weboldal konfigurációját saját domain névvel (vagy alias-szal) a LAN környezetben. Megértik a `ServerName` és `ServerAlias` használatát, illetve a hosts fájl szerepét a névfeloldásban belső hálózaton. Cél továbbá a logfájlok szétválasztása virtual hostonként és további Apache direktívák (pl. külön error log útvonal) alkalmazása.
- **Előfeltételek:** Apache web szerver fut, a hallgató tud alap config fájlt szerkeszteni. A Windows hosts fájl módosításához admin jog a host gépen. Az eddigi laborokon kialakított környezet (statikus weboldal saját könyvtárban) előny, de tiszta lappal is kezdhető a gyakorlat.

Tanári magyarázat és demonstráció

VirtualHost fogalma: Az Apache lehetővé teszi, hogy egy fizikai szerver (vagy VM) egyszerre több webhelyet szolgáljon ki, akár külön domain neveken. Ezt nevezzük virtuális host konfigurációnak. Két típusa van: - **Name-based VirtualHost:** A hostnév alapján különbözteti meg a kéréseket. (A böngésző a Host fejlécben küldi a domain nevet, Apache ez alapján választ configot.) Ez a gyakoribb. - **IP-based vagy port-based VirtualHost:** A szerver IP-címe vagy a port alapján különülnek el a site-ok. (Pl. egy webszerver két IP-vel rendelkezik, mindegyiken más site fut, vagy port 80-on az egyik, port 8080-on a másik site.)

Mi a neve alapú megoldást használjuk. LAN-ban nincs publikus DNS-ünk, de a **hosts fájl** segítségével tudunk kreálni tetszőleges neveket.

Példa forgatókönyv: Legyen a szerveren két webhely: 1. **alapértelmezett (vagy első) weboldal**, mondjuk a tanuló saját oldalát, ami eddig a /home/student/weboldal_demo alatt futott. Ezt kössük egy névhez, pl. `diak.local`. 2. **második weboldal**, lehet egy kitalált cég honlapja vagy projekt. Ennek külön könyvtárat hozunk létre és külön konfigurációt, pl. `projekt.local` néven.

Természetesen mindkettő ugyanazon a szerveren fut, ugyanazon a 80-as porton, a böngésző a nevek alapján kapja a megfelelőt. Ehhez beállítjuk a VirtualHost-okat.

Új site könyvtár létrehozása: Hozzunk létre egy mappát a /var/www vagy /home/student alatt a második weboldalnak, pl.:

```
sudo mkdir -p /var/www/projekt.local/public_html
```

(Itt a Linuxize ajánlása egy strukturált elrendezés domain szerint[47][48]. Mi is használhatjuk ezt a sémát a demo kedvéért.) Tegyünk bele egy index.html-t, hogy lássuk majd:

```
echo "<h1>Üdv a PROJEKT oldalán!</h1>" | sudo tee
/var/www/projekt.local/public_html/index.html
```

Adjuk meg a megfelelő jogosultságokat:

```
sudo chown -R www-data:www-data /var/www/projekt.local
sudo find /var/www/projekt.local -type d -exec chmod 755 {} \;
sudo find /var/www/projekt.local -type f -exec chmod 644 {} \;
```

(Ez csak azt biztosítja, hogy a webszerver olvasni tudja. Mivel /var/www alatt van, ott a /var/www valószínű 755, olvasható, tehát nincs plusz home jogosultsági trükk.)

Virtuális host konfiguráció létrehozása: A Debian-ban a szokás, hogy minden site kap egy saját conf fájlt a /etc/apache2/sites-available-ben. Nevezzük el a domain után, pl. `projekt.local.conf`. Hozzuk létre:

```
sudo nano /etc/apache2/sites-available/projekt.local.conf
```

Tartalomnak a következőt írjuk:

```
<VirtualHost *:80>
    ServerName projekt.local
    ServerAlias www.projekt.local
    DocumentRoot /var/www/projekt.local/public_html

    <Directory /var/www/projekt.local/public_html>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/projekt_error.log
    CustomLog ${APACHE_LOG_DIR}/projekt_access.log combined
</VirtualHost>
```

Magyarázat: - `<VirtualHost *:80>`: minden interfészen a 80 porton érkező kérésekre érvényes, ha a hostnév egyezik[49]. - `ServerName projekt.local`: ez a fő domain név a site-hoz[50]. - `ServerAlias www.projekt.local`: aliasként a www-s változat is ide tartozik[50]. - `DocumentRoot`: a webfájlok helye (amit előbb hoztunk létre)[50]. - `<Directory ...>`: a könyvtár engedélyezése. Beállítottuk, hogy ne listázza a könyvtárat, ha nincs index (`-Indexes`), és engedjük a symlinkeket. **AllowOverride All**: itt demonstráljuk, hogy ennél a site-nál engedélyezzük a .htaccess használatát (mert pl. lehet majd egy PHP alapú cucc). Ez különbözik az előző site beállításától, ahol None volt. Természetesen, hogy .htaccess működjön, mod_rewrite már engedélyeztük a múltkor. - `ErrorLog` és `CustomLog`: külön fájlba írja ennek a site-nak a logjait, így nem keveredik a többivel[51][52]. (Ha nem adjuk meg, mehetne közösbe, de ez jobb átlátás miatt.)

Mentsük el.

Virtuális host engedélyezése és a default letiltása: Használjuk az a2ensite parancsot:

```
sudo a2ensite projekt.local.conf
```

Ekkor kiírja, hogy symlink létrehozva a sites-enabled-ben[30]. Mivel a 000-default.conf még mindig aktív, és abban nincs ServerName, az alábbi helyzet áll elő: ha Host header nincs vagy nem egyezik másikra, akkor a default szolgál ki. De mi szeretnénk, hogy a diák saját eddigi oldala (amit talán 000-defaultban módosítottunk) is kapjon egy nevet. Egyszerűen beállíthatnánk a 000-defaultnak egy ServerName-et pl. diak.local és átnevezhetnénk a file-t diak.local.conf-ra, stb., vagy csinálhatjuk az lusta módszerrel: marad default, de a hosts fájlban majd diak.local név nélkül, csak IP-vel hivatkozzuk. Haladóbb: csináljuk rendesen: - Hozzunk létre egy `diak.local.conf`-ot a sites-available-ben, hasonlót a projekt.local-hoz, DocumentRoot-tal a diák home-os mappájára (vagy /var/www/diak.local/public_html ha áttesszük). - Engedélyezzük azt is: `sudo a2ensite diak.local.conf`. - Tiltsuk le a defaultot: `sudo a2dissite 000-default.conf`[53].

Ha idő szűkös, maradhat a 000-default és a projekt.local együtt. Az Apache a host header alapján a megfelelő VirtualHost blokkot választja, ha talál olyat, aminek ServerName vagy

Alias egyezik. Mivel a default-nak nincs, a konkrét névre a projekt.local config megy, minden másra (pl. IP alapon vagy diak.local-nak nem tudja melyik legyen) a default.

Hosts fájl beállítása a kliens oldalon: A Windows gépeken (vagy ahol el akarjuk érni a site-ot) hozzá kell adni a domain neveket a hosts fájlhoz, hogy a mi szerver IP-nkre mutassanak. Tegyük fel a szerver IP-je 192.168.1.100, akkor a tanulók a saját PC-jük `C:\Windows\System32\drivers\etc\hosts` fájljának végéhez hozzáadják:

```
192.168.1.100   diak.local
192.168.1.100   www.projekt.local projekt.local
```

(Mindkét domain a mi szerverünkre mutat.) Ezután a böngészőben: - **http://diak.local/** behívva a diák saját oldalának kell megjelennie. - **http://projekt.local/** pedig a másik projekt oldalt jeleníti meg. A **www.projekt.local** is menni fog, mert aliasnak beállítottuk.

Ezzel szimuláltuk a többszörös webhely kiszolgálást.

Ellenőrzés és hibaelhárítás: - Indítsuk újra az Apache-ot a site engedélyezések után: `sudo systemctl reload apache2`. - A diák nézheti a `sites-enabled` könyvtárat, látja a symlinkeket. - Az `apachectl -S` parancs hasznos: kiírja az aktív VirtualHostokat, a hozzájuk tartozó ServerName-eket, és melyik config fájlból jönnek. (Mutassuk meg, szépen listáz.) - Ha a projekt.local behívásakor a default oldal jön be, az azt jelenti, hogy nem találta meg a configot. Oka: vagy nem engedélyeztük (a2ensite hiány), vagy a böngésző cache miatt (de az kevésbé, domain változott). Vagy a hosts bejegyzés hiányzik, és valódi DNS nincs, ilyenkor a request el sem jut a mi szerverünkre.

DirectoryIndex tiltás hatásának bemutatása: Az Options -Indexes miatt ha valaki beírja http://projekt.local/valami/ és nincs index file, a szerver 403-at ad. Ezt ellenőrizhetjük, vagy be is állíthatjuk modulárisan. (Ez inkább mellékes most.)

Diákfeladatok lépésről lépésre

1. **Második webhely mappa létrehozása:** A diákok hozzanak létre a saját szerverükön egy második weboldal könyvtárat. Használhatják a /var/www struktúrát domain szerint, pl.:

   ```
   sudo mkdir -p /var/www/masodik.oldal.hu/public_html
   sudo chown -R $USER:$USER /var/www/masodik.oldal.hu
   ```

   (Itt $USER a saját nevük, így először a saját userüknek adják, hogy fájlt tudjanak bele tenni, utána majd adunk www-data-nak is.) Tegyenek bele tartalmat:

   ```
   echo "<h2>Masodik oldal tartalma</h2>" >
   /var/www/masodik.oldal.hu/public_html/index.html
   ```

   Majd adják át www-data-nak és 755/644 jogosultságot, ahogy gyakoroltuk:

   ```
   sudo chown -R www-data:www-data /var/www/masodik.oldal.hu
   sudo find /var/www/masodik.oldal.hu -type d -exec chmod 755 {} \;
   sudo find /var/www/masodik.oldal.hu -type f -exec chmod 644 {} \;
   ```

2. **Virtuális host konfiguráció készítése:** Másolják le a korábban bemutatott sablont:

```
sudo nano /etc/apache2/sites-available/masodik.conf
```

Tartalom például:

```
<VirtualHost *:80>
    ServerName masodik.oldal.hu
    DocumentRoot /var/www/masodik.oldal.hu/public_html
    <Directory /var/www/masodik.oldal.hu/public_html>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/masodik_error.log
    CustomLog ${APACHE_LOG_DIR}/masodik_access.log combined
</VirtualHost>
```

(Lehet alias is, de nem kötelező most.) Mentsék el. Futassanak egy `sudo apachectl configtest`-et – ha Syntax OK, akkor:

3. **Site engedélyezése és default letiltása:**

```
sudo a2ensite masodik.conf
sudo a2dissite 000-default.conf
sudo systemctl reload apache2
```

(Ha valaki az első site-nak is csinált külön confot pl. elso.conf, akkor azt is a2ensite, a defaultot pedig mindenképp lőjük ki, hogy ne zavarjon.)

4. **Hosts fájl szerkesztése a kliensen:** A diák a **saját gépén** (Windows) adjon hozzá egy hosts bejegyzést a domain-hez. Például:

```
192.168.1.100   masodik.oldal.hu
192.168.1.100   elso.oldal.hu
```

(A megfelelő IP-vel és nevekkel, amiket használtak. Ahol $USER.homework.local vagy ilyesmi.)

5. **Tesztelés böngészőben:** Most a **http://masodik.oldal.hu/** URL-nek a második site tartalmát kell adnia, **http://elso.oldal.hu/** pedig az elsőt. Ha működik, látják mindkét külön h1/h2-t. Kísérletezhetnek: mi történik, ha IP-cím alapján mennek a szerverre? (Valószínűleg a „első” oldalt kapják, mert az Apache a default vhostot választja, ami most lehet hogy a mi elso.oldal.hu config lett ha nem tiltottuk le, de ha letiltottuk a defaultot és csak névhez kötött van, akkor az első definíció lesz a default. Az `apachectl -S` megmondja.)
6. **Logfájlok ellenőrzése:** Nézzék meg a `/var/log/apache2/masodik_access.log` tartalmát: látniuk kell benne a masodik.oldal.hu hostra érkező kéréseket. Ugyanígy az error.log-ot, bár remélhetőleg üres. Ezzel igazolják, hogy külön logolódik a site. (Akár generálhatnak egy 404-et csak a masodik site-n egy random URL-lel, és látják, hogy a hiba is az ő logjába megy.)

7. **Optional – alias és www:** Ha van idő, beállíthatják, hogy a masodik site a www prefixszel is működjön: tegyenek `ServerAlias www.masodik.oldal.hu` sort a conf-ba, reload Apache, és hosts fájlba is vegyék fel a www-s nevet. Utána próbálják ki böngészőben. (Ez közelíti a való élet domain alias működést.)

Tipikus hibák és megoldásaik

- **Nem jó VirtualHost választódik ki:** Ha a böngészőben nem azt a tartalmat látjuk, amit vártunk, lehet:
- Böngésző cache: használjunk inkognitó módot vagy ctrl+F5, de domain váltásnál általában nem gond.
- Nem jól szerkesztettük a hosts fájlt, és a név nem arra az IP-re mutat. Pingeljük a nevet Windows CMD-ben: `ping masodik.oldal.hu` – a válasz IP-t nézzük. Ha nem a mi szerver IP-nk, akkor a hosts nincs jó helyen szerkesztve (talán elfelejtették admin módban, vagy elírták a file nevet – figyelni, hogy hostsnak nincs kiterjesztése).
- Elfelejtették engedélyezni a site configot vagy újraindítani az Apache-ot. Nézzük `apachectl -S` – kiírja mely vhostok aktívak. Ha a mienk nincs listában, akkor kimaradt az a2ensite vagy nem reloadoltak. Pótolni kell.
- Ha IP-vel nyitják meg és nem domainnel, mindig az első (default) VirtualHost lesz kiszolgálva. Esetünkben, ha kikapcsoltuk a 000-default-ot, akkor valószínű a config fájl név sorrendje dönt (Debianban az a2ensite linkek előtti szám alapján). Ha elso.conf és masodik.conf van, lehet abc sorrend. Ezt ne zavarja meg őket, domainnel kell használni.
- **ServerName ütközések:** Ha véletlen két configban is ugyanaz a ServerName, Apache figyelmeztet a restartnál, hogy duplikált vhost (és azt hiszem az elsőt preferálja). Kerüljük az ilyet, mindegyik egyedi legyen.
- **Nem működik a www alias:** Ha a `ServerAlias` kimaradt, a www.domain név host bejegyzés hiába van, Apache nem társítja a site-hoz, így valószínű a default vhost lesz. Megoldás: adjuk hozzá az alias-t, reload.
- **Permission hiba a második site-nál:** Ha valaki nem /var/www alá, hanem pl. a home könyvtárába tette a második site-ot, el ne felejtse a home 711 trükköt. Ugyanaz érvényes rá, mint az előző órán. Itt modulárisan érdemes megbeszélni: Sokan mondják, hogy websitet a /var/www alá, mert annak a /var-nak a jogosultsága globális olvasható. A home-nál extra lépés kell.

Összefoglalás és kérdések

Összegzés: Ezen a foglalkozáson a hallgatók képessé váltak több webhely beállítására egy Apache szerveren. Elsajátították a VirtualHost konfigurációk kezelését, és látták, hogyan lehet *egyedi domain nevek* alapján szétválasztani a forgalmat. Megtanulták, hogyan lehet a kliens gépen (fejlesztői gépen) a hosts fájl segítségével elérni a nem publikus domain neveket is. A logok testreszabásával megkönnyítik a több site adminisztrációját. Ezek az ismeretek jól jönnek, ha pl. egy fejlesztői szerveren több projekt fut egyszerre, vagy a későbbiekben ha élesben is hostolnak több domaint egy szerveren.

Kérdések: - Mi az a VirtualHost és miért van rá szükség? Hogyan választja ki az Apache, hogy melyik VirtualHost konfigurációt használja egy bejövő kéréshez?
- Milyen Apache parancs(ok)kal lehet egy új virtuális host konfigurációt aktiválni vagy deaktiválni?
- Hová érdemes tenni a különböző weboldalak állományait a fájlrendszerben, és miért?
- Mi a feladata a `ServerName` és `ServerAlias` direktíváknak? Hogyan tudod tesztelni a működésüket LAN környezetben, ahol nincs saját DNS szerver?
- Hogyan lehet elérni, hogy két külön weboldal naplóadatai ne keveredjenek? (Milyen utasításokkal állítanád be?)
- Ha hozzáadsz egy új VirtualHost configot, melyik parancs mutatja meg, hogy sikerült-e betölteni és mik az aktív hostok?

---

6. foglalkozás (45 perc): **mod_userdir és WordPress telepítése MariaDB-vel, tesztelés LAN-on**

Oktatási cél és előfeltételek

- **Cél:** Bemutatni a mod_userdir használatát, amellyel minden felhasználó saját weboldalt szolgáltathat a `~felhasználó` URL alatt. Ezen keresztül a diákok láthatják egy többfelhasználós webszerver modelljét. Továbbá, a foglalkozás fő gyakorlata egy **WordPress tartalomkezelő rendszer telepítése** a frissen kialakított LAMP környezetre. A diákok létrehoznak egy MariaDB adatbázist és felhasználót a WordPress számára, konfigurálják a wp-config.php-t, és helyi hálózaton elérhetővé teszik a WordPress oldalt. Megtanulják a WordPress telepítő használatát és a gyakori buktatókat (fájljogosultság, mod_rewrite/.htaccess engedélyezése) megoldani.
- **Előfeltételek:** A korábbi foglalkozások eredménye: működő Debian webszerver Apache+PHP-FPM-mel, futó MariaDB adatbázisszerver telepítve (ha még nem, most telepítjük). Alapvető SQL ismeret előny (CREATE DATABASE és felhasználó). A diákok legyenek tisztában a WordPress fogalmával (miért ezt telepítjük, mi az a CMS). Internetkapcsolat szükséges a WordPress letöltéséhez (alternatív megoldás: előre letöltött zip a LAN-on).

Tanári magyarázat és demonstráció

mod_userdir – Felhasználói weboldalak könyvtára: A mod_userdir Apache modul segítségével minden Unix/Linux felhasználó készíthet weboldalt a saját home könyvtárában egy előre meghatározott mappában (pl. `public_html`). Az URL formája: `http://szervernev/~felhasznalo/`[54], és az adott user public_html könyvtárának tartalmát szolgálja. Ez jól jöhet oktatási környezetben vagy webhosztingnál, ahol mindenkinek adnak egy kis helyet weboldalnak. Alapértelmezésben Debianban a modul nincs engedélyezve a biztonság kedvéért (mert több user tartalmai egy domain alatt futnak, ami hordoz kockázatokat[55]).

mod_userdir engedélyezése:

```
sudo a2enmod userdir
sudo systemctl restart apache2
```

(A modul betöltődik, és a `/etc/apache2/mods-enabled/userdir.conf` konfiguráció életbe lép, ami alapból azt mondja: `UserDir public_html`, vagyis a `~user` kéréseket a /home/user/public_html felé irányítja.) Ezzel *bármely rendszer user* akinek van public_html könyvtára, kiteheti a webjét.

Teszteljük a mod_userdir-t: Hozzunk létre a *student* felhasználónknak egy public_html-t:

```
mkdir ~/public_html
echo "Hello, Userdir World!" > ~/public_html/index.html
```

Mivel a /home/student alapesetben /home/student:700 (privát) vagy 750, ezt fel kell oldani, különben az Apache nem tud belépni. Emlékezzünk:

```
chmod 711 /home/student
chmod 755 ~/public_html
```

(Igen, a userdir.conf tartalmaz is utalást, hogy a modul default inaktív volt régen, és hogy a root dir-t tiltani ajánlott – ezzel ne foglalkozzunk most. A lényeget csináljuk.) Most nyissuk meg böngészőben: **http://<szerver-ip>/~student/**. Meg kell jelennie a "Hello, Userdir World!" szövegnek. Ezzel bizonyítottuk a modul működését. (Ha nem jelenne meg: nézzük error.log. Valószínű permission denied, ha kimaradt a chmod 711. Ezt a mod_userdir doksi is kiemeli, hogy meg kell adni a más számára execute-ot, különben 403[56].)

Röviden beszéljük meg a kockázatot: mod_userdir-rel a user kódja ugyanúgy fut, mintha a fő oldalon lenne, tehát ha pl. PHP-t futtat ott, a webszerver user (www-data) alatt megy. Egymás adataihoz nem fér hozzá a user (ha open_basedir nincs korlátozva, elvileg hozzáférhetne más user public_html-jéhez is). Valamit valamiért, ez egy egyszerű megoldás multiuser webszerverre.

WordPress CMS – bevezető: A WordPress a legnépszerűbb tartalomkezelő rendszer (CMS) a világon, PHP-ben íródott, és egy MySQL/MariaDB adatbázist használ háttérként. Rengeteg bővítmény, téma elérhető hozzá. A telepítése tipikusan: 1. LAMP környezet előkészítése (**L**inux, **A**pache, **M**ySQL/MariaDB, **P**HP) – ezt már megtettük. 2. Üres adatbázis létrehozása a WP-nek, és egy felhasználó, aki csak ahhoz fér. 3. WordPress fájlok letöltése (wordpress.org-ról) és bemásolása a webszerver egy könyvtárába. 4. WordPress konfigurálása: a `wp-config.php` kitöltése (adatbázis elérhetőségek, salted keys stb.). 5. Webes telepítő futtatása: beállítani a webhely nevét, admin felhasználót.

MariaDB adatbázis telepítése (ha nem lenne): Debian 12 esetén a `mariadb-server` csomag telepítendő:

```
sudo apt install mariadb-server
```

Ez elindítja a MariaDB-t. Futtassuk a biztonsági beállító scriptet:

```
sudo mariadb-secure-installation
```

Itt a következőket érdemes:

- Set root password? Igen, adjunk meg egy jelszót az adatbázis root userének. - Remove anonymous users: igen.

- Disallow root login remotely: igen (biztonság).

- Remove test database: igen.

- Reload privileges: igen.

Ezzel kész az alap config. Jegyezzük meg: a MariaDB parancs sor használatához mostantól kell a -p (mert jelszó van). Illetve a Debiannál van unix_socket auth is, de ezt hagyjuk.

WordPress adatbázis és felhasználó létrehozása: Jelentkezzünk be a MariaDB konzolba:

```
sudo mariadb -u root -p
```

(kéri a jelszót, adjuk meg, belép.) Adjuk ki az SQL utasításokat:

```
CREATE DATABASE wordpress CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'jelszo123';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Magyarázat: Létrehoz egy "wordpress" nevű adatbázist (utf8mb4 Unicode – WP-nél ajánlott)[57], létrehoz egy wpuser nevű felhasználót csak a localhostról való belépésre erős jelszóval[57], megad neki minden jogot a wordpress DB minden táblájára[57], majd frissíti a jogokat. (A flush privileges amúgy nem muszáj, a GRANT is flush-el). Ez a bevett gyakorlat: külön user a WP-nek, nem a root-tal futtatjuk.

WordPress letöltése és kihelyezése: Lépjünk egy ideiglenes könyvtárba és szerezzük be a legfrissebb WP csomagot:

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xvzf latest.tar.gz
```

Ettől lesz egy `wordpress` könyvtár tele fájlokkal (wp-config-sample.php, wp-content, stb.)[58]. Ezt a könyvtárat kell átmásolni a webszerverünk egy elérhető helyére. Ha az előző órán a diák csinált egy VirtualHost-ot kifejezetten WP-nek, beteheti oda. Ha nem, akkor használhatjuk az alap DocumentRoot-ot is. (Tegyük fel, hogy a diák csinált pl. egy `projekt.local` site-ot, ami statikus volt. Ezt felhasználhatjuk a WP-hez.) Másoljuk a fájlokat:

```
sudo rsync -avP wordpress/ /var/www/projekt.local/public_html/
```

(A példában a projekt.local alatti public_html-t használjuk. Ha a diák más pathre szeretné, mondjuk /var/www/wordpress, csinálhatnak új confot is. Egyszerűség kedvéért vegyük, hogy a "második weboldal" virtuális hostjukat a WP-hez fogják felhasználni.)

Jogosultságok beállítása WP-hez: Fontos, hogy a teljes WordPress fájlstruktúra a webszerver által írható lesz néhány helyen (pl. wp-content/uploads). Minimálisan adjuk meg:

```
sudo chown -R www-data:www-data /var/www/projekt.local/public_html
```

(Debianon általában elég, ha a www-data a tulaj, mert uploadkor a futó PHP (www-data) userként tud írni. Bizonyos telepítő plugin esetén 775 folder is kellhet, de amíg ugyanaz a user, addig 755/644 is elég.) A biztonság kedvéért a telepítés előtt:

```
sudo find /var/www/projekt.local/public_html/ -type d -exec chmod 755 {} \;
sudo find /var/www/projekt.local/public_html/ -type f -exec chmod 644 {} \;
```

Ezzel read-onlyvá tettük a PHP számára a fájlokat, kivéve ahol kell majd (de feltöltéskor generálja magának). **Megjegyzés:** Egyes WP útmutatók lazábbra veszik (777 upload-ra, stb.), de mi inkább maradjunk a minimál jogoknál, és ha gond, majd állítanak azon.

WordPress konfiguráció – wp-config.php: A WordPress alapban a `wp-config-sample.php` fájlt adja, ezt kell másolni:

```
cd /var/www/projekt.local/public_html
sudo cp wp-config-sample.php wp-config.php
sudo nano wp-config.php
```

Szerkesszük a kritikus részt:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'jelszo123' );
define( 'DB_HOST', 'localhost' );
```

Más változtatás most nem kell, de felhívjuk a figyelmet a biztonsági kulcsokra: van egy link generálásra[59]. Jó gyakorlat lenne betenni egyedi auth salt kulcsokat a configba, de nem kötelező a telepítéshez. (Most időhiány miatt ezt kihagyhatjuk, de elmondjuk, hogy a https://api.wordpress.org/secret-key/1.1/salt/ címen generálható, és a kapott define sorokat be kell másolni a megfelelő helyre[59].)

WordPress telepítő futtatása böngészőben: Nyissuk meg: **http://projekt.local/** (a domain persze a hosts fájlban feloldva a szerver IP-re). Ekkor a WordPress felismeri, hogy még nincs telepítve (de a config file már megvan, DB elérhető), ezért egy telepítő oldal jön. Itt végigmenni: - Válasszunk nyelvet (magyar). - Adjunk meg Site Title (pl. "Projektem"), Admin felhasználó nevet, jelszót, email címet[60]. - Telepítés gomb.

Ha minden jól ment, a WP bejelentkező oldalára dob át, ahol az admin beléphet.

WordPress tesztelése: Lépjünk be az admin felületre (wp-login.php vagy /wp-admin/). Hozzunk létre egy próba bejegyzést, töltsünk fel egy képet (ez utóbbi azért is, hogy lássuk, megy-e a file feltöltés, ha nem, mi a hiba – tipikusan a php.ini upload_max_size korlát vagy az uploads könyvtár jogosultság). Nyissuk meg a honlapot (http://projekt.local/) és ellenőrizzük, hogy a tartalom megjelenik.

Friendly URL beállítása (Permalinks): Alapértelmezésben a WP a paraméteres URL-t használja (pl. ?p=1). Ha szebb URL-t akarunk (Pl. /2025/11/hello-vilag/), akkor be kell kapcsolni a "Permalink" beállításnál valami strukturát. Az adminban menjünk Settings -> Permalinks, válasszuk mondjuk a "Post name" opciót, mentsük. **Itt jön a csavar:** Ehhez a WP .htaccess-be ír, ami `AllowOverride All`-t igényel a vhost configban és mod_rewrite modul meglétét. Mi ezt megtettük (AllowOverride All be volt a vhost configban, mod_rewrite is engedélyeztük). Ha nem lenne, a WP figyelmeztetne pirossal, hogy nem tud írni .htaccess-be, vagy a szerveren nem megy az URL átírás. Mivel nálunk rendben, a .htaccess létre is jön magától, tartalom: WordPress rewrite szabályok.
Teszt: a poszt linkjére kattintva szép URL-el betölt. Ha 404-et kapnánk, akkor a hiba mod_rewrite hiány vagy .htaccess tiltás. Ezt megbeszéljük és javítjuk (a2enmod rewrite, vhost AllowOverride All, Apache reload).

WordPress LAN-on való megosztása: Mivel a projekt.local domain nem publikus DNS, a diák gépén hosts-ban oldottuk. Ha más is meg szeretné nézni, akinél nincs benne, nem fogja tudni. Ilyenkor mondhatjuk a tanár gépére is beírjuk hosts-ba, hogy megnézze, vagy a diák megoszt egy linket IP-vel (de WP-nél az base URL domainhez kötött, szóval IP-vel nem biztos minden működik rendesen). Ezért javasoljuk: akinek meg akarják mutatni, az is vegye fel hosts-ba a nevet. Vagy egy alternatíva: be lehet állítani, hogy a WP dinamikusan jó legyen IP-vel is, de az bonyolítja. Maradjunk annyiban: a WP belső beállítása (WordPress Address, Site Address) fix domain. Ha azt IP-címre állítanák, menne IP-vel, de akkor domainnel nem. Ezért marad a hosts szerkesztése a nyerő, ha máshol is be akarunk lépni.

Diákfeladatok lépésről lépésre

1. **mod_userdir kipróbálása:** Engedélyezzék a modult: `sudo a2enmod userdir && sudo systemctl restart apache2`. Készítsenek a saját felhasználójuk home-jában public_html mappát: `mkdir ~/public_html`, és tegyenek bele egy index.html-t. Adják meg a jogosultságot: `chmod 711 ~ && chmod 755 ~/public_html`. Ezután a host gépen nyissák meg: http://<IP-cím>/~<felhasználó>/. Látniuk kell az index tartalmát. (Ha nem, nézzék meg az Apache error.log-ot a „permission denied /~user" sorokat[61] – ha ilyet látnak, a chmod 711 kimaradt. Javítsák és nézzék újra.)
2. **MariaDB adatbázis beállítás:** Telepítsék, ha még nincs: `sudo apt install mariadb-server`. Futtassák a `mariadb-secure-installation`-t és állítsanak be root jelszót, töröljék a test DB-t, anon usert. (Jegyezzék fel a root jelszót maguknak, el ne felejtsék.)
   Lépjenek be a konzolra: `sudo mariadb -u root -p`.

3. Hozzanak létre egy `wordpress` nevű adatbázist:

```
CREATE DATABASE wordpress DEFAULT CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

4. Hozzanak létre egy `wpuser` nevű usert (saját jelszóval, pl. `Wp@12345`):

```
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'Wp@12345';
```

5. Adjanak minden jogot az új usernek az új DB-re:

```
GRANT ALL ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

   Jegyezzék fel ezt a db nevet, usert, jelszót.

6. **WordPress letöltése:** A diák menjen a webszerver gépén egy tmp-be és `wget`-elje a WP-t, majd csomagolja ki. Alternatív: ha nincs internet a VM-ben, a tanár adhat a megosztott mappán egy wordpress.zip-et, akkor kibontja. Tegyük fel a wget működik. Parancsok:

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzvf latest.tar.gz
```

   Ekkor van egy `/tmp/wordpress` könyvtár tele fájlokkal.

7. **WordPress fájlok elhelyezése:** Döntsék el, hová telepítik. Lehet a korábbi masodik oldal helyére. Pl. aki csinált `masodik.oldal.hu` vhostot, azt használjuk:

```
sudo rsync -avP /tmp/wordpress/ /var/www/masodik.oldal.hu/public_html/
```

   (Megkérdezhetjük, van-e külön vhost. Ha valaki default site-ra akarja, lehet /var/www/html-be is másolni, vagy home-os weboldal_demo-ba, de egyszerűbb egy külön vhost.) Utána állítsák be a jogosultságot:

```
sudo chown -R www-data:www-data /var/www/masodik.oldal.hu/public_html
```

   (Esetleg a chmod find-os parancsokat is, de ha már korábban 755/644 volt és a cp megtartotta, nem muszáj. Az rsync -a lehet 7750-es jött, de chown után a group persmission is ok.)

8. **wp-config.php beállítása:** Menjenek a WP dirbe, másolják a config sample-t:

```
sudo cp wp-config-sample.php wp-config.php
sudo nano wp-config.php
```

   Töltsék ki a DB adataikat:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
```

```
define( 'DB_PASSWORD', 'Wp@12345' );
define( 'DB_HOST', 'localhost' );
```

Mentsék el.

9. **Telepítő futtatása:** Biztosítsák, hogy a vhost configban a DocumentRoot jó helyre mutat. (Ha nem vhostot használnak, akkor a default DocumentRoot-ra tették. Lényeg, hogy http://... elérjük.) Nyissák meg böngészőben a domain nevét. El kell induljon a WP telepítő. Válasszanak nyelvet (magyar), majd töltsék ki:
10. Webhely címe (pl. Saját Blog),
11. Felhasználó (admin, de jelszót ne hagyják "gyenge" szinten, adjanak erőset vagy pipálják be, hogy elfogadják a gyengét),
12. E-mail. Telepítés indul. Ha kész, bejelentkező link jelenik meg.
13. **WordPress használatának próbalépései:** Lépjenek be az admin felületre (wp-admin). Írjanak egy új bejegyzést "Helló Világ" tartalommal, publikálják. Nézzék meg a honlapot (főoldalt), megjelenik-e. Töltsenek fel egy képet a bejegyzésbe (Media upload). Ha a feltöltés sikertelen, figyeljék a hibaüzenetet. Gyakori ok:
14. php.ini limit kicsi (2M), de mi ezt 16M-re állítottuk, szóval egy képhez valószínű elég. Ha mégis limit error, utaljunk vissza: php.ini upload_max_size.
15. uploads könyvtár írhatósága: mivel az egész WP dir www-data tulaj, tud a PHP oda írni. Ha root lenne, nem tudna. De mi chownoltuk, úgyhogy megy.
16. mod_security vagy hasonló modul beleszól, nálunk nincs ilyen. Tehát várhatóan a kép feltöltése sikeres. Meg is jelenik a bejegyzésben.
17. **Permalink beállítás teszt:** Adminban, Settings->Permalinks, válasszanak "Post name" opciót, mentsék. Böngészőben nyissák meg a posztot újra (most /hello-vilag/ formában lesz). Ha betölt, akkor mod_rewrite működik és .htaccess engedve. Ha 404-et kapnak, akkor:
18. Nézzék meg a /var/www/..../public_html/.htaccess fájlt. Létrejött-e? Ha nem, valószínűleg nem tudta megírni, talán hiányzik a AllowOverride All.
19. Nézzék meg az Apache error.log-ot, van-e rewrite hiba. Valószínűbb, hogy kimaradt a `AllowOverride All` a vhost Directory részéből. Ezt javítsák, majd `sudo systemctl reload apache2`. Utána működni fog. (A WP is figyelmeztetett volna a Permalinks oldalon, ha a .htaccess nem írható).
20. **WordPress elérés LAN-on:** Kérjék meg diáktársukat vagy a tanárt, hogy a WP oldalukat a saját gépéről nyissa meg. Ezt csak úgy tudja, ha felveszi a hosts fájlba a domain-t (ugyanúgy, ahogy ők is). Tegyenek egy próbát: (Ez optional, ha van idő). Ha a tanár gépén megnyílik a WP oldal, gratuláljunk – belső hálón futó webszervert hoztak létre, amin egy népszerű webalkalmazás fut. (Fel lehet vetni: mi kéne, hogy interneten is menjen? Nyilván publikus IP és DNS, plusz port forward stb., de ezt már túlmutat a tananyagon.)

Tipikus hibák és megoldásaik

- **WordPress telepítő nem éri el az adatbázist:** Ha a telepítő oldalon azt írja "Error establishing a database connection", akkor a megadott DB adatok hibásak. Lehet elírt jelszó a wp-config-ban, vagy nem fut a MariaDB.
- Nézzék a `sudo systemctl status mariadb` (ha nem fut, indítsák).
- Lépjenek be `mariadb -u wpuser -p wordpress` parancscsal, megnézni beenged-e ezzel a userrel. Ha nem, akkor user vagy pass rossz.
- Esetleg a DB host nem localhost lenne (nálunk de). Javítás után újratölteni a telepítő oldalt.
- **WP telepítés utáni fehér oldal / PHP error:** Ha valami plugin gond lenne (nincs, mert még nem raktak), vagy hiányzó PHP modul. Pl. a WP alapnak kell a php-xml modul (a core-hoz?), de általában a `php` meta csomag felrakta azt is. Ha mégis hiányozna modul (pl. mbstring), a WP oldalán hibaüzenet jelezné. Telepíteni kell a hiányzó php8.2-* csomagot, FPM restart.
- **WP adminban furcsa karakterek:** Ha locale/charset gond lenne, de mivel utf8mb4-et használtunk, jónak kell lennie. Ha valaki nem állította be, és alap latin1 lett a DB, akkor lehetnek "?" karakterek. Ezt nehéz utólag javítani, de remélhetőleg megelőztük.
- **Nem működik a Permalink (404):** Ezt kitárgyaltuk: mod_rewrite modul engedélyezve van-e és Directory AllowOverride All be van-e kapcsolva[22]. Ha kimaradt, pótolják és Apache reload. WP .htaccess-e is legyen ott (ha WP nem tudta létrehozni, akkor kézzel másolják be a WP által javasolt rewrite szabályokat, de jobb adni neki írási jogot átmenetileg és save permalinks megint).
- **Képfeltöltés nem működik:** Ha WP media uploadnál "Could not create directory wp-content/uploads/..." hiba van, akkor jogosultsági gond: a `wp-content/uploads` könyvtár létrehozását próbálta, de nem sikerült. Ok: a `wp-content` könyvtár valamiért nem www-data tulajdon (vagy sticky?). Mi mindent www-data-ra állítottunk, de ha valaki kihagyta, pótolja: `sudo chown -R www-data:www-data wp-content`. Esetleg open_basedir restrikció, de azt nem állítottunk be. Ha a hiba "Exceeds the maximum upload size", akkor a php.ini limit kevés. Ezt már 16M-re tettük, ha valaki ennél nagyobb képet tölt, mondjuk egy 20MB-os, akkor előjön. Megoldás: php.ini-ben tovább emelni.
- **mod_userdir korlátozás**: Említhetjük: ha nem akarjuk, hogy minden user használhassa, a userdir.conf-ban lehet `UserDir disabled` és `UserDir enabled username` paramétereket megadni[62][63]. Alap biztonság miatt root tiltva is van. Ha valaki ~root-ot próbálná, az mod_userdir configban tiltott[64], 403-al vissza is utasítja.

Összefoglalás és kérdések

Összegzés: Az utolsó foglalkozás során a tanulók **teljes LAMP alkalmazást** telepítettek: a WordPress-t. Ezzel integráltan használták az Apache webszervert, PHP-t és MariaDB-t. A

telepítés rávilágított az adatbázis létrehozás és jogosultság-kezelés fontosságára, a fájlok és könyvtárak jogosultságaira (webszerver írhatósága a tartalomkezelőnél), valamint a webes alkalmazások tipikus konfigurációs lépéseire (pl. URL átírás engedélyezése). Emellett megismerték a mod_userdir modult, ami többfelhasználós környezetben érdekes – ezzel látták, hogy egy webszerver egyszerre kiszolgálhat *felhasználói weblapokat* és *központi webalkalmazásokat* is, más-más útvonalakon. A WordPress sikeres beüzemelése a LAN-on egy szép lezárása a gyakorlati résznek: innentől a diákok eljátszhatnak a WordPress testreszabásával (témák, bővítmények), amivel mélyítik a megértést a webszerver és PHP modul kapcsolatáról is (pl. egy cache plugin igényelhet külön modult, stb.).

Kérdések: - Mi a mod_userdir modul célja és milyen URL formátumot használ? Hogyan lehet korlátozni, hogy ne minden felhasználó használhassa?
- Sorold fel a WordPress telepítés fő lépéseit egy LAMP szerveren (milyen komponenst kell előkészíteni, milyen fájlokat hova kell másolni, mit kell beállítani)!
- Miért nem tanácsos root felhasználóval futtatni egy webalkalmazást adatbázis kapcsolattal? Mit tettünk ennek érdekében a WP telepítésnél?
- Milyen PHP modulok (kiterjesztések) szükségesek a WordPress hibamentes futtatásához? (Gondolj a MySQL, XML, képkezelésre stb.) Hogyan ellenőrizheted, hogy telepítve vannak-e?)
- Hogyan biztosítottuk, hogy a WordPress telepítő képes legyen létrehozni a konfigurációs fájlt és a .htaccess-t? (Milyen könyvtár/jogosultság beállításokkal?)
- Ha a WordPress oldalon a „szép linkek” (permalinks) nem működnek, milyen Apache-beállítás(oka)t kell ellenőrizni és esetleg módosítani?
- Bonus: Milyen további lépéseket tennél, ha ezt a WordPress oldalt élesben, interneten kellene futtatni? (Pl. SSL bekapcsolása, biztonsági bővítmények, rendszeres frissítés, stb.)

Összefoglaló konfigurációs fájl lista

Az alábbiakban összegyűjtjük a legfontosabb konfigurációs fájlokat és helyeket, melyeket a gyakorlat során módosítottunk vagy használunk:

- **/etc/netplan/** – pl. `01-netcfg.yaml`: statikus IP hálózati beállítása Debian 12 alatt (netplan konfiguráció)[65][8].
- **/etc/apache2/apache2.conf** – Apache fő konfigurációs fájl, betölti a mods-enabled, sites-enabled stb. állományokat[15][16].
- **/etc/apache2/ports.conf** – Apache hallgató portok beállítása (alapesetben „Listen 80”, valamint 443 ha engedélyezik SSL modult)[17].
- **/etc/apache2/sites-available/** – Virtuális host definíciók könyvtára, pl. `000-default.conf`, `projekt.local.conf`, `masodik.conf`. Innen az a2ensite parancs symlinkeli őket a sites-enabled alá[66][30].

- **/etc/apache2/sites-enabled/** – Aktív (engédélyezett) site configok (szimbolikus linkek). Ezeket olvassa be Apache induláskor[18].
- **/etc/apache2/mods-available/** – Apache modul leírók (modul betöltés *.load és konfig* .conf fájlok). pl. `rewrite.load`, `php8.2-fpm.conf`, `userdir.conf`[19].
- **/etc/apache2/mods-enabled/** – Engedélyezett modulok linkjei. a2enmod/a2dismod kezeli[19].
- **/etc/apache2/conf-available/** – Általános konfiguráció töredékek, pl. `php8.2-fpm.conf` itt található (ez tartalmazza a FastCGI Handler beállítást a .php fájlokra)[37][39].
- **/etc/apache2/conf-enabled/** – Engedélyezett config töredékek (a2enconf kapcsolja).
- **/etc/apache2/mods-enabled/userdir.conf** – A mod_userdir beállításai (pl. `UserDir public_html`, tiltva root számára)[54][64].
- **/etc/php/8.2/fpm/php.ini** – A PHP fő konfigurációja (FPM mód). Itt állítottuk be: `display_errors`, `upload_max_filesize`, `post_max_size`, `date.timezone` stb.[67].
- **/etc/php/8.2/fpm/pool.d/www.conf** – PHP-FPM „pool" konfiguráció. Itt definiált a processz menedzsment (pm.max_children, stb.) és a socket path (listen)[42][43].
- **/etc/php/8.2/mods-available/** – PHP modul konfigurációk. Pl. `mysqli.ini`, `gd.ini` stb. A phpenmod parancs innen linkel az fpm/conf.d alá.
- **/etc/mysql/mariadb.conf.d/** – MariaDB szerver beállítások könyvtára. (Általában ide tartozik a 50-server.cnf, ahol az bind-address, sql_mode stb. beállítható. Ehhez most nem kellett nyúlni.)
- **C:\Windows\System32\drivers\etc\hosts** (Windows oldalon) – A lokális gép névfeloldási listája. Ide vettük fel a saját domain neveinket a szerver IP-címéhez, pl. `projekt.local`[5][6].

PHP-FPM fő beállítási pontok

A PHP-FPM működésével kapcsolatos legfontosabb beállítások és tudnivalók:

- **PHP-FPM szolgáltatás és pool:** A php-fpm úgy működik, hogy előre létrehoz N darab PHP interpretáló folyamatot (worker), és az Apache ezen folyamatok egyikének küldi a PHP feldolgozását. A paraméterek a **www.conf** file-ban:
- `pm = dynamic` – dinamikus folyamatszám, automatikusan igazodik.
- `pm.max_children` – legfeljebb hány PHP worker folyamat futhat egyszerre (e fölött sorba áll a kérés)[68].
- `pm.start_servers`, `pm.min_spare_servers`, `pm.max_spare_servers` – induló folyamatok száma, minimum és maximum üresjáratú folyamat. Pl. start_servers=2, min_spare=1, max_spare=3, így terhelés szerint 1-3 idle maradhat, felette leállítja a fölösleget.
- `pm.max_requests` – hány kérést szolgáljon ki egy worker, mielőtt újraindul (memory leak-ek enyhítésére).

- **Listen socket/port:** A PHP-FPM alapértelmezetten Unix domain socketen figyel: `listen = /run/php/php8.2-fpm.sock`[42]. Ezt az Apache config `SetHandler` utasításában használtuk is. Szükség esetén átállítható TCP portra (pl. 127.0.0.1:9000), de a socket gyorsabb lokálisan. Ha portot használnánk, Apache configban is azt kéne megadni[69][70].
- **FPM user/group:** A www.conf-ban: `user = www-data`, `group = www-data`[71] – tehát a PHP folyamatok a webszerver felhasználó jogán futnak. Ezért nem volt gond, hogy a fájlok www-data tulajdonban vannak. Ha több pool lenne (pl. több felhasználónak elkülönítve), lehetne userenként más.
- **PHP ini fontos opciók:** Fejlesztés során:
- `display_errors = On` – hogy a hibák megjelenjenek a böngészőben (ne csak logban). Éles rendszeren Off[67].
- `log_errors = On` – (alapértelmezett) a hibákat az error.log-ba írja. Fejlesztésnél is hasznos.
- `memory_limit` – ne legyen túl alacsony (WP-nek min. 128M ajánlott).
- `upload_max_filesize`, `post_max_size` – fájlfeltöltési korlátok, pl. 16M-re állítottuk (alap 2M)[44].
- `date.timezone` – mindig állítsuk be a pontos zónát, különben E_WARNING megy a logba időkezelésnél[67].
- **PHP modulok betöltése:** Debian csomagokkal telepített modulok automatikusan betöltődnek (ini file linkelve). Ha saját modult adunk, a php.ini-ben az `extension=modulename.so` bejegyzéssel vagy a mods-available conf útján tehetjük meg. A `php -m` paranccsal listázhatók a betöltött modulok, vagy phpinfo() böngészőben.
- **Kommunikáció Apache és FPM között:** Az Apache error.log-ban tipikusan így látszik egy FPM hiba: `AH01079: failed to make connection to backend: httpd-UDS` – ez arra utal, hogy a socket elérés sikertelen (FPM nem fut, vagy permission gond). Ilyenkor ellenőrizni: fut-e php-fpm (`systemctl status`), illetve a socket fájl jogosultsága (`ls -l /run/php/...sock`). Nálunk a socket owner www-data, group is www-data[72], Apache is www-data, ezért ok. Ha mismatched, a `listen.owner` és `listen.group` paraméterekkel lehet állítani (alapból www-data)[73].
- **Restart vs Reload:** PHP-FPM-nél a `systemctl reload php8.2-fpm` megpróbál elegánsan újraindulni (graceful), de gyakoribb a sima restart. Apache reload nem érinti a php-fpm folyamatokat, csak új configot olvas. Minden php.ini módosítás után **kötelező** restartolni az FPM-et, különben a változás nem lesz aktív[74].
- **One FPM, multiple sites:** Jelenleg egy FPM "pool" szolgálja ki az összes weboldalt. Ez egyszerű, de ha pl. különböző site-oknak más php beállítást akarunk (más memory_limit, user), létrehozhatunk több pool-t, külön Unix sockettel és külön Apache SetHandler-rel vhostonként[75][76]. Erre példa: futtathatnánk egy külön poolt más userrel a mod_userdir-es oldalakhoz. Ez azonban már haladó konfiguráció, most nem volt rá szükség.

Hasznos parancsok gyűjteménye

Az alábbi parancsok és beállítások voltak a legfontosabbak a gyakorlatok során, érdemes ezeket áttekinteni és megjegyezni:

- **Rendszer és hálózat:**
- `ip addr show` – hálózati interfészek és IP-k listázása (statikus IP beállítás előtt/után ellenőrzés)[77].
- `sudo netplan apply` – Netplan hálózati konfiguráció érvényesítése statikus IP beállításakor[78].
- `ping <cél>` – hálózati kapcsolat tesztelése (pingeljük a gateway-t vagy más hostot) a kapcsolat ellenőrzéséhez[79].
- `sudo ufw allow 'Apache'` – UFW tűzfalon Apache profil engedélyezése (80-as port nyitása)[27].
- `sudo ufw status` – tűzfal státuszának ellenőrzése, nyitott portok listája.

- **Apache webszerver:**

- `sudo systemctl status apache2` – Apache szolgáltatás státusz lekérdezése (fut-e, van-e error).
- `sudo systemctl start|stop|restart apache2` – Indítás/leállítás/újraindítás.
- `sudo systemctl reload apache2` – Konfiguráció újratöltése (graceful restart, nem bontja a kapcsolatok).
- `sudo apachectl configtest` vagy `sudo apache2ctl -t` – Konfiguráció szintaxis ellenőrzése indulás előtt[32].
- `sudo apachectl -S` – VirtualHost konfigurációk összegzése (megmutatja, mely hostnevek mely file-hoz tartoznak, default jelöléssel).
- `a2enmod <modul>` / `a2dismod <modul>` – Apache modul engedélyezése/letiltása (pl. `a2enmod rewrite` a mod_rewrite-hez, `a2enmod userdir`)[80].
- `a2ensite <site-conf>` / `a2dissite <site-conf>` – Site konfiguráció engedélyezése/letiltása (pl. `a2ensite projekt.local.conf`, majd reload)[30][53].
- `a2enconf <name>` / `a2disconf <name>` – Általános config töredék (conf-available) engedélyezése (pl. `a2enconf php8.2-fpm` – a PHP-FPM Apache integráció bekapcsolása)[46].
- `tail -f /var/log/apache2/access.log` – Apache hozzáférési napló valós idejű követése (ctrl+c-vel kilépés)[25].
- `tail -f /var/log/apache2/error.log` – Apache error log követése, hibakereséshez nélkülözhetetlen[81][24].

- **MySQL/MariaDB adatbázis:**

- `sudo mariadb-secure-installation` – MariaDB kezdeti biztonsági beállítás (root jelszó, stb.)[82].

- `sudo mariadb -u root -p` – MariaDB konzol indítása rootként (Debiannál a sudo nélkül is megy socket auth-tal, de mi jelszót adtunk).
- SQL utasítások adatbázisban:
  - `SHOW DATABASES;` – adatbázisok listája.
  - `CREATE DATABASE <name> [CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci];` – új adatbázis létrehozása WP-hez (utf8mb4 a modern emoji-khoz is kell)[57].
  - `CREATE USER 'user'@'localhost' IDENTIFIED BY 'pass';` – új felhasználó létrehozása csak local kapcsolat esetén[57].
  - `GRANT ALL PRIVILEGES ON <dbname>.* TO 'user'@'localhost';` – jogosultság adása a usernek a megadott DB-re[57].
  - `FLUSH PRIVILEGES;` – jogok érvényesítése (Grant után nem mindig szükséges, de jó gyakorlat)[83][84].
  - `EXIT;` – kilépés a SQL konzolból.
- `sudo systemctl status mariadb` – Adatbázis szerver státusz ellenőrzése (ha valamiért nem tud csatlakozni WP, ezzel kezdjük).
- `mysqldump -u <user> -p <database> > export.sql` – (Csak említés szintjén, hogy lehet DB-t menteni, nem gyakoroltuk, de hasznos admin parancs.)

- **Linux fájlkezelés (a webes környezethez kapcsolódó):**

- `sudo cp <src> <dst>` – másolás (pl. wp-config-sample.php -> wp-config.php).
- `sudo mkdir -p <dir>` – könyvtár(ak) létrehozása (a -p a köztes könyvtárakat is létrehozza, pl. /var/www/site/public_html)[85].
- `sudo chown -R www-data:www-data <könyvtár>` – rekurzív tulajdonosváltás webszerver userre (tipikusan, hogy Apache/PHP írni tudja)[28].
- `sudo chmod -R 755 <könyvtár>` – rekurzív jogosultság állítás: könyvtárak 755, fájlok 755 (ha -R vakon megy, a fájlokat is 755-tel látja el, ami nem gond, de a fentebb bemutatott find-os módszer precízebb).
- `find <könyvtár> -type d -exec chmod 755 {} \;` – minden könyvtárra 755 adása[28].
- `find <könyvtár> -type f -exec chmod 644 {} \;` – minden fájlra 644 adása. Ez a kettő együtt a webes fájloknál biztonságos default (fájl írható ownernek, olvasható mindenkinek; könyvtár beléphető mindenkinek).
- `ln -s <target> <linkname>` – szimbolikus link létrehozása (Apache a2ensite is ezt csinálja a háttérben).
- `wget <URL>` – fájl letöltése weben keresztül (pl. WordPress tar.gz)[58].
- `tar -xvzf file.tar.gz` – tar.gz kicsomagolása (x: extract, v: verbose, z: gzip, f: file megadása)[58].

- `rsync -avP <src> <dest>` – fájlok és könyvtárak szinkron másolása, megőrzi a jogosultságokat/időbélyegeket (a archív mód), v verbose, P progress. Ezt használtuk WP fájlok másolására gyorsan[86].
- `echo "text" | sudo tee file.txt` – egy sor gyors beírása fájlba root-ként. Pl. weboldal gyors tartalom létrehozásnál használtuk.

- **Hasznos egyéb:**

- `sudo nano <file>` – fájl szerkesztése szöveges felületen (nano editor).
- `grep -R "keresett" /etc/apache2` – adott kifejezés keresése configokban (hasznos, ha nem tudjuk, hol van pl. egy modul neve megemlítve).
- `php -v` – PHP verzió ellenőrzése (CLI).
- `php -m` – betöltött PHP modulok listája (CLI).
- `php -a` – PHP interaktív mód (shell), ritkán hasznos webszervernél, de van.
- `wp-cli` – (Nem használtuk, de megemlítjük) Parancssori WordPress vezérlő eszköz, amivel admin műveletek scriptelhetők. Fejlett téma, de adminoknak hasznos.

Ezek a parancsok és beállítások alkotják a Linux alapú webszerver üzemeltetésének eszköztárát. Az órák során mindegyiket használtuk valamilyen formában. Gyakorlással rögzülnek, és a későbbiekben bármilyen LAMP stack vagy webszolgáltatás beállításánál jól jönnek. Aki magabiztosan tudja alkalmazni őket, az képes lesz önállóan telepíteni és konfigurálni egy webszervert és tipikus webalkalmazásokat Debian/Linux környezetben.